Pietro Farina

Firewall de Aplicação: Utilizando Árvore Sintática Para Prevenção de Ataques Por Injeção

Brasil

2024, v-1

Pietro Farina

Firewall de Aplicação: Utilizando Árvore Sintática Para Prevenção de Ataques Por Injeção

Monografia apresentada na disciplina Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharelem Ciência da Computação.

Centro Universitário Senac

Bacharelado em Ciência da Computação

Orientador: Dr. Eduardo Heredia

Brasil

2024, v-1

Agradecimentos

Agradeço à minha família, que sempre estiveram ao meu lado, lidando comigo sempre com muito carinho e paciência e me apoiando em minha trajetória.

Ao professor orientador Eduardo Heredia por todas as discussões tanto sobre assuntos relacionados ao trabalho, quanto assuntos gerais sobre segurança da informação. Nossas conversas foram importantes para moldar minha percepção sobre a área.

Aos meus professores, que me ensinaram e ajudaram a me desenvolver para chegar até aqui.

Aos meus amigos da faculdade e de fora dela, que sempre discutiam comigo assuntos relacionados aos tópicos de segurança, e tecnologia da informação no geral. Essas interações sempre foram muito importantes para mim.

Resumo

Com o crescimento exponencial da internet e seu uso para o armazenamento e tráfego de dados sensíveis e de alto valor, cibercriminosos buscam explorar falhas de segurança em sistemas vulneráveis para comprometer e obter vantagens indevidas com esses dados. Entre as falhas de segurança mais exploradas até hoje está a injeção de SQL. Essa vulnerabilidade permite que invasores manipulem e encadeiem consultas SQL diretamente no banco de dados de sistemas comprometidos, possibilitando o acesso não autorizado a informações confidenciais ou o comprometimento da integridade dos dados.

Embora diversas técnicas de detecção de ataques tenham surgido, elas raramente são eficientes em todos os cenários. Diante desse contexto, este trabalho propõe o uso da análise sintática como uma abordagem para detectar tentativas de injeção de SQL em requisições HTTP, visando proteger sistemas vulneráveis contra a exploração desse tipo de falha.

Palavras-chaves: WAF. Análise Sintática. Injeção de SQL.

Abstract

With the exponential growth of the internet and its increasing use for storing and transmitting sensitive and high-value data, cybercriminals seek to exploit security vulnerabilities in systems to compromise and gain unauthorized advantages from this exposed data. Among the most commonly exploited vulnerabilities is SQL injection. This flaw allows attackers to manipulate and chain SQL queries directly into the database of compromised systems, enabling unauthorized access to confidential information or compromising data integrity.

Although various attack detection techniques have been developed, they are rarely effective in all scenarios. In this context, this work proposes the use of syntactic analysis as an approach to detect SQL injection attempts in HTTP requests, aiming to protect vulnerable systems from the exploitation of this type of flaw.

Key-words: WAF. Syntactic Analysis. SQL Injection.

Lista de ilustrações

Figura 1 – Fluxo de um ataque de XSS 22
Figura 2 – Fluxo do ataque de SQLI 23
Figura 3 – Técnicas de acesso inicial 2023. 25
Figura 4 – Principais motivações por trás de crimes ciberéticos 26
Figura 5 – Custo médio do comprometimento por área de atuação. 27
Figura 6 – Ilustração, firewall de Aplicação 30
Figura 7 – Response to the question, “What best describes your organization’s approach to WAF?” 34
Figura 8 – Decomposição da expressão em tokens 36
Figura 9 – Exemplo de árvore sintática. 37
Figura 10 – Arquitetura do proxy reverso 41
Figura 11 – Testando a API com sqlmap 53
Figura 12 – Tentativa de extração das tabelas com SQLMap 54
Figura 13 – Logs de acesso da API sobre a tentativa de ataque com SQLMap . . . 55

Lista de tabelas

Tabela 1 – Grandes ataques XSS e seus impactos 28
Tabela 2 – Performace do ModSecurity 34
Tabela 3 – Padrões que geraram loops 56
Tabela 4 – Requisições não respondidas 56

Lista de abreviaturas e siglas

WAF	Web Application Firewall
XSS	Cross Site Scripting
SQLi	Injeção de SQL
WEB	World Wide Web
HTML	HyperText Markup Language
CDN	Content Delivery Network

Sumário

1 INTRODUÇÃO . . . 17
1.1 Contexto . . . 17
1.2 Objetivo . . . 18
1.3 Objetivo Principal . . . 18
1.4 Objetivo Específico . . . 18

2 REVISÃO BIBLIOGRÁFICA . . . 19
2.1 As vulnerabilidades . . . 19
2.2 Tipos de Vulnerabilidades . . . 20
2.2.1 Entrada de Dados não Verificada . . . 21
2.2.2 Cross Site Scripting . . . 22
2.2.3 SQL Injection . . . 23
2.2.4 Logging Insuficiente . . . 24
2.3 Ataques Cibernéticos . . . 24
2.4 Ataques de Injeção . . . 25
2.4.1 Impactos . . . 26
2.4.2 Mitigações contra ataques de Injeção . . . 28
2.4.3 *Firewall* de Aplicação . . . 29
2.4.3.1 White-Listing . . . 30
2.4.3.2 Uso de correspondência de padrões . . . 31
2.5 Falso-Positivo e Falso-Negativo . . . 33
2.6 Análise Gramatical . . . 35
2.6.1 Análise Léxica . . . 35
2.6.2 Análise Sintática . . . 36
2.6.3 Análise Semântica . . . 37

3 DESENVOLVIMENTO . . . 39
3.1 Escolha Da Gramática . . . 39
3.1.1 Escolha Dos Comandos Disponíveis . . . 39
3.2 Escolha Do Tipo de Parser . . . 40
3.3 Arquitetura . . . 40
3.4 Escolha Da Linguagem . . . 41
3.5 Analisador . . . 42
3.5.1 O Lexer . . . 42
3.5.1.1 As estruturas . . . 42
3.5.1.2 Identificação da sequência . . . 42

3.5.1.3 Definição do tipo 43
3.5.1.4 Funções De Interface 44
3.5.1.5 Verificação de Comentário e Espaços 44
3.5.2 O Parser 44
3.5.2.1 A Estrutura Parser 45
3.5.2.2 Funções De Interface 45
3.5.2.3 Análise da Expressão 45
3.6 O Firewall De Aplicação 46
3.6.1 Técnica de Análise 47
3.6.1.1 Consulta Genérica 47
3.6.2 Verificando Uma Manipulação Na Consulta 48
3.6.2.1 Funções De Interface 49
3.6.2.2 Verificação de Uma *Query String* 49

4 TESTES E RESULTADOS 51
4.1 Aplicação Web Vulnerável 51
4.1.1 Teste Na API Com SQLMap 52
4.2 Identificando Explorações Simples 53
4.2.1 Teste SQLMap com WAF 54
4.2.2 Fuzzing Com Lista Padrões Maliciosos 55

5 CONCLUSÃO 59
5.1 Trabalhos Futuros 59

REFERÊNCIAS 61

APÊNDICE A – LISTA DE PADRÕES 67
A.1 Apendice A - Lista para Teste de *Fuzzing* 67

1 Introdução

1.1 Contexto

Aplicações WEB sempre foram alvo de tentativa de exploração por parte de agentes maliciosos, visando tomar alguma vantagem em cima dos dados que esse sistema guarda, ou mesmo abusar de seus recursos. Conforme os estudos dirigidos pela OWASP expõem, ataques de injeção continuam entre as técnicas mais usadas para o comprometimento de aplicações web. Como a edição de 2021 do estudo sugere, as falhas que envolvem controles de acesso mal desenvolvidos se tornaram o maior vetor de ataque da atualidade (OWASP..., 2021). A edição de 2023 da Top Ten Bugs Surfaced, estudo feito pela empresa de *bug-bounty* HackerOne, mostrou que ataques de injeção estiveram em primeiro lugar quando analisando relatórios de pesquisadores publicados na plataforma e, em segundo lugar, em testes de intrusão dirigidos por especialistas durante o período que antecede a publicação (HACKER1..., 2023). Esse tipo de problema afeta a aplicação web vulnerável e seu público de diversas maneiras, sendo uma das mais comuns o roubo de dados dos seus usuários. Dentro do grupo dos ataques de injeção existe a categoria da injeção de SQL, que explora vulnerabilidades em consultas de banco de dados em web sites, permitindo a execução de comandos maliciosos e o acesso não autorizado a informações sensíveis. Isso coloca tanto os dados dos usuários quanto a integridade da aplicação em risco. Injeção de SQL é normalmente um dos principais vetores buscados por criminosos digitais.

Como forma de impedir ataque que se baseiam em injeção, como injeção de SQL, diversos sistemas empregam o uso de *firewalls* de aplicação para filtragem de anomalias (SHAHRIAR, 2018). Essas soluções analisam o tráfego com base em assinaturas de ataques já verificados no passado. Estudos sugerem que a melhor abordagem para se concluir que o padrão é malicioso ou não, é com a junção de listas de padrões conhecidos (*blacklists*) e *pattern matching* (BELLO, 2020). No entanto, diversos testes, geralmente conduzidos por cibercriminosos, são feitos visando a descoberta de padrões capazes de degradar a segurança do sistema, de forma que esses mecanismos de defesa não consigam detectar a ação e frequentemente obtendo êxito nessa tarefa, pois *firewalls* de aplicação baseados em assinatura não são capazes de prever todas as possibilidades. Isso torna a manutenção das regras de WAF (*Web Application Firewall*) algo complexo e, em alguns casos, inviável, pela elevação nas taxas de falsos-positivos quando se tenta criar um sistema abrangente, uma variável que irá mensurar o quanto a experiência do usuário pode ser afetada pelas mitigações.

1.2 Objetivo

Este trabalho tem como objetivo propôr um algoritmo para a detecção e bloqueio de ataques que visam explorar vetores de injeção de SQL com base em análise sintática.

1.3 Objetivo Principal

O estudo aqui descrito pretende desenvolver um algoritmo baseado em análise recursiva descendente, visando propor um método nele baseado, para a detecção do abuso de SQL em ataques de injeção. Neste caso, a própria gramática de um subgrupo do SQL é utilizada para verificar se a linguagem está sendo utilizada para explorar uma vulnerabilidade existente.

A abordagem proposta busca definir de maneira mais fiel à gramática da linguagem que está sendo explorada maliciosamente, permitindo assim descobrir com mais precisão os padrões potencialmente prejudiciais para o sistema.

O analisador criado será uma versão simplificada da gramática da comumente usada linguagem de consulta SQL, que seja o suficiente para testar a eficiência dessa aproximação baseada em análise sintática em determinados casos de uso.

1.4 Objetivo Específico

- Criar um Lexer para um tipo genérico para um subgrupo de SQL.
- Criar um parser que interaja com o Lexer para analisar sequências de caracteres.
- Criar um algoritmo capaz de analisar tráfego de rede HTTP, e fazer avaliações com base no parser.

2 Revisão Bibliográfica

O objetivo deste capítulo é desenvolver a base de conceitos fundamentais e necessários para o entendimento do projeto. Neste capítulo, serão discutidos e fundamentados temas como ataques cibernéticos, ataques por vetores de injeção, mitigações e firewalls de aplicação. Posteriormente, haverá uma introdução à análise sintática, que servirá como base para proposta de solução de problemas relacionados ao firewall de aplicação comum.

2.1 As vulnerabilidades

Uma vulnerabilidade é uma característica não intencional em um sistema de informação, que pode ser explorada para comprometer sua segurança, afetando o bom funcionamento desse sistema. Uma vulnerabilidade pode surgir a partir de uma falha no design, mau funcionamento devida à implementação incorreta daquilo que foi planejado no desenvolvimento do sistema, ou pode surgir a partir de uma degradação na qualidade do código (OWASP, 2023; ITEAM, 2020).

Em (E. E. Schultz Jr; D. S. Brown; T. A. Longstaff, 1990), menciona-se que uma vulnerabilidade é uma característica, ou um bug, que permite ao usuário ultrapassar medidas impostas de segurança. Portanto, torna o sistema exposto e propenso a ser utilizado de formas não previstas, usualmente maliciosamente.

> O protocolo IP não foi projetado para o ambiente atual da internet. Embora projetado para ser tolerante a falhas de hardware, o IP não possui muita resistência contras ataques intencionais. Atualmente, a única maneira segura de proteger os pacotes contra espionagem é utilizando criptografia. (STARLIN, 1998)

O texto acima foi retirado do livro **TCP/IP Internet - Intranet - Extranet** e mostra como o tempo tornou uma tecnologia vulnerável. Isso porque, a internet foi projetada sem as intenções daquilo que traz seus usos atuais, que até então, servia às comunidades restritas. Portanto, não foram desenvolvidos mecanismos integrados para manter o sigilo e a integridade do dado (STARLIN, 1998). Hoje o protocolo é exposto a diversas vulnerabilidades, como a falsificação do número de sequência, ataques de roteamento, falsificação de endereço de origem e os vários ataques de autenticação que podem surgir em decorrência da falta de encapsulamento seguro do dado (BELLOVIN, 1989; HARRIS; HUNT, 1999).

Existe uma noção de que sistemas com muitas funcionalidades dão abertura para que estas sejam usadas e exploradas de forma maliciosa. Por exemplo, o sistema operacional **Windows** oferece diversas funcionalidades para se comunicar com outras máquinas na rede, transferir arquivos e até autenticação remota. Isso fornece ao atacante, a capacidade de pular a outras máquinas conectadas na rede, desde que elas se comuniquem entre si, conforme os protocolos definidos. Esse tipo de situação gera uma vulnerabilidade, não por mau funcionamento, mas por uma má utilização. Além disso, um ambiente com sistemas usando tecnologias e protocolos em comum, terão vulnerabilidades em comum (E. E. Schultz Jr; D. S. Brown; T. A. Longstaff, 1990).

2.2 Tipos de Vulnerabilidades

Com o surgimento de estudos, vulnerabilidades foram sendo catalogadas, atribuindo a elas propriedades que definem seu tipo, categoria e o meio de se explorar. Existem muitos tipos de vulnerabilidades como erro humano; implantadas em um sistema por um agente malicioso que já possui acesso; decorrência de outros fatores, como a descoberta de falhas e bugs. Caso um software dependa de uma biblioteca que esteja com código vulnerável, esse software pode chegar a herdar a vulnerabilidade (SIMON, 2001; E. E. Schultz Jr; D. S. Brown; T. A. Longstaff, 1990).

A OWASP (Open Web Application Security Project), uma organização global sem fins lucrativos dedicada a promover a segurança da informação, com foco principal em aplicações web, por meio de metodologias de auditoria, correção e mitigação de vulnerabilidades, oferece ferramentas, documentos e boas práticas que auxiliam empresas e desenvolvedores a identificar e reduzir riscos de segurança em seus sistemas e aplicações (OWASP, 2023). A OWASP menciona alguns exemplos de vulnerabilidades comuns no contexto de aplicações web:

- Falta de validação de entrada de dados do usuário
- Falta de um mecanismo de registro (logging) suficiente
- Tratamento de erros em modo fail-open
- Não fechamento adequado da conexão com o banco de dados

Assim como a listagem acima, este projeto estará focando em vulnerabilidades que afetam o contexto de aplicações WEB.

2.2.1 Entrada de Dados não Verificada

A vulnerabilidade de *falta de validação de entrada de dados do usuário*, vulnerabilidades de entrada não verificada ou **Improper input validation** (no inglês) ocorre quando os dados são inseridos pelo usuário (via formulários) mas não validados ou validados incorretamente, sem fazer todos os testes para ter certeza se a entrada está conforme o necessário para ser processada (MITRE, 2006; SYNACK, 2024).

A exploração desse tipo de vulnerabilidade pode resultar na modificação do fluxo de execução, e lidar eventualmente com a execução de códigos maliciosos, com um potencial prejudicial ao usuário ou ao sistema (MITRE, 2006).

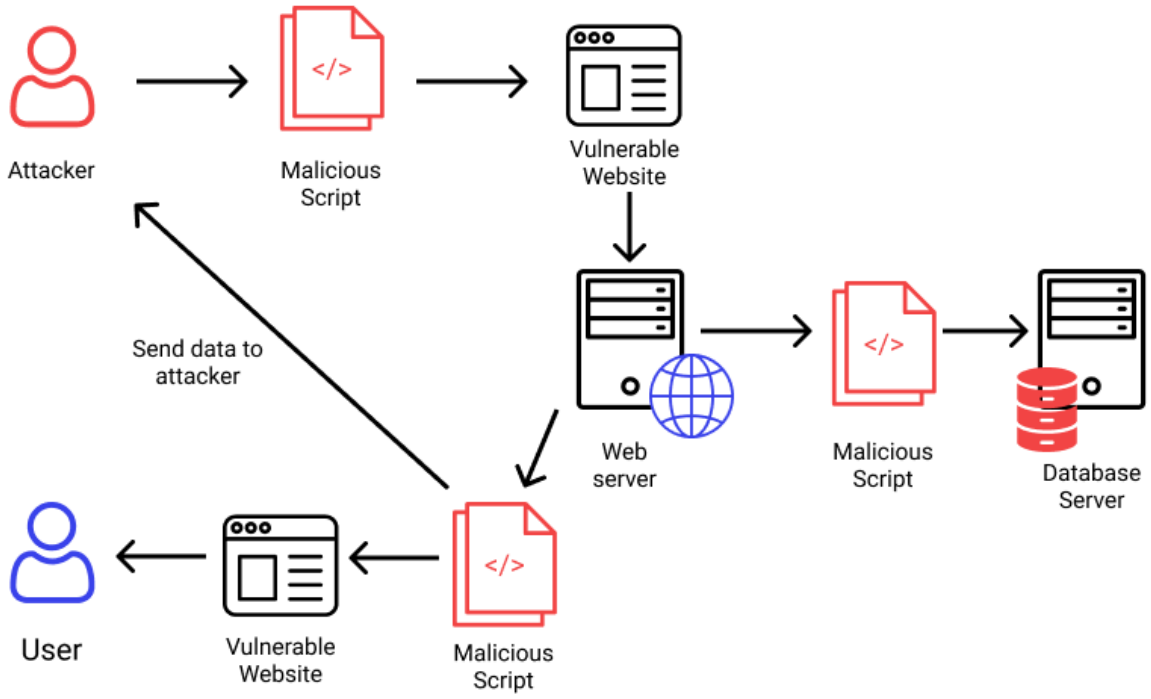
2.2.2 Cross Site Scripting

Um exemplo é a exploração de XSS (Cross Site Scripting (MITRE, 2014)), categorizada na classe de falhas de injeção pela OWASP (OWASP..., 2021). Essa é uma das implicações mais comuns da vulnerabilidade de entrada não verificada, e tem impactos críticos na segurança do usuário (ILYIN, 2024).

A vulnerabilidade ocorre quando há uma entrada de dados controlada pelo usuário que é renderizada na página. Se essa entrada não for devidamente verificada, um agente malicioso pode inserir código-fonte, códigos em JavaScript e código HTML. Quando carregados na página, esses elementos são interpretados como parte do código-fonte original da página web. Como o interpretador não consegue distinguir entre o código da página web e aquilo que foi inserido pelo usuário, ele acaba interpretando e executando a entrada do usuário (ILYIN, 2024; MITRE, 2014; GUPTA, 2015).

Caso a entrada de dados feita por um usuário A seja mostrada a um usuário B, se houver a vulnerabilidade, o usuário A pode executar códigos no navegador do usuário B assim que o dado é solicitado e interpretado pelo seu navegador. A Figura 1 mostra o fluxo de exploração da vulnerabilidade.

Figura 1 – Fluxo de um ataque de XSS



Fonte: (ILYIN, 2024)

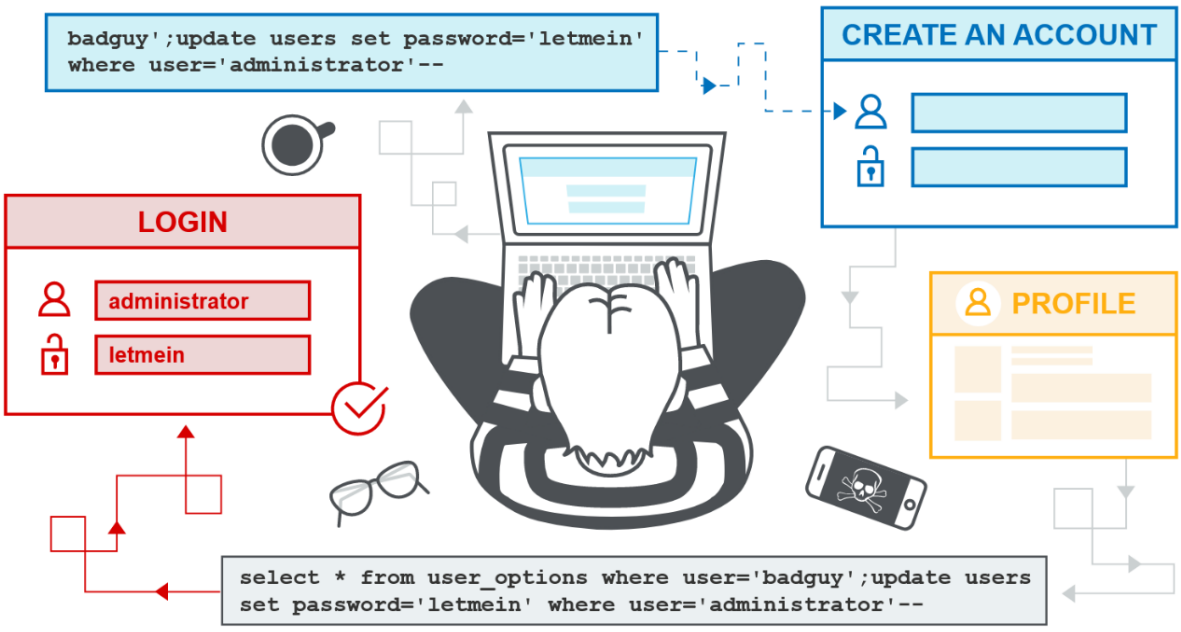
O usuário A, representado como o atacante, envia um script (em Javascript) por meio de um formulário vulnerável. Esse formulário pode ter a funcionalidade de salvar comentários em uma publicação, alterar o nome do usuário ou adicionar uma descrição

a um produto. O dado malicioso injetado pelo atacante é enviado ao servidor web e armazenado no banco de dados. Quando outro usuário (usuário B) acessa a página de comentários, por exemplo, essa informação, o script malicioso, é solicitada, inserida na página e executada no navegador do usuário (ILYIN, 2024). Como impacto, deve ser possível fazer o navegador na vítima tomar ações, e até roubar a sessão da vítima no site que ela esteja utilizando. Em posse da sessão do usuário, o atacante tem acesso aos seus dados pessoais (MITRE, 2014).

2.2.3 SQL Injection

Outra falha comum, frequentemente citada ao lado de XSS, é a falha de SQLi, ou Injeção de SQL. SQLi é categorizada como uma falha de injeção pela OWASP (OWASP..., 2021). Quando uma aplicação web é vulnerável a essa falha, um agente malicioso pode interagir com o banco de dados por meio de formulários expostos ao problema. A vulnerabilidade ocorre porque muitos desenvolvedores concatenam comandos SQL com a entrada do usuário sem realizar a devida validação (validação de que os dados estão estruturados conforme o esperado). Isso permite que, além dos dados esperados, o usuário consiga inserir comandos SQL adicionais, encadeando instruções no fluxo de execução original. Como resultado, o atacante pode manipular as consultas SQL executadas pelo servidor, potencialmente obtendo acesso não autorizado a dados sensíveis, modificando informações ou até mesmo comprometendo a integridade do banco de dados.

Figura 2 – Fluxo do ataque de SQLI



Fonte: (ITRC, 2023b)

A Figura 2 ilustra a exploração de uma vulnerabilidade de SQLi. Na imagem, o

usuário tem acesso a um formulário de autenticação da aplicação que está vulnerável. O usuário mal-intencionado insere o seguinte comando: *badguy';UPDATE users SET password='letmein' WHERE user='administrator'–*. O formulário deveria receber apenas um nome de usuário, *badguy*, e uma senha, mas devido à vulnerabilidade, o comando SQL adicional é executado.

O comando encadeado altera a senha do usuário *administrator* para uma senha escolhida pelo atacante, *letmein*. Essa ação permite que o atacante acesse a conta do administrador da aplicação web, pois agora ele conhece a nova senha do administrador (ITRC, 2023b).

2.2.4 Logging Insuficiente

Toda aplicação web deve *logar*, ou seja, salvar cada interação do usuário. Esse dado pode ser importante para auditorias futuras, tanto para resolver problemas quanto para traçar o desenvolvimento de um ataque. Caso não haja o armazenamento do uso das funções, isso se torna uma vulnerabilidade, pois um possível abuso será ofuscado e poderá passar despercebido (MITRE, 2009).

A OWASP cita que teve dificuldade na pesquisa desse tipo de vulnerabilidade e desvendar a frequência que ela é encontrada nos sistemas, pois se não houve dificuldade concreta na utilização desses dados, na necessidade dos logs, se dificulta saber se todos os recursos estão sendo monitorados, e se o monitoramento abrange todas as possibilidades do recurso. Então ter a informação da relação de sistemas vulneráveis envolveria uma análise mais minuciosa (OWASP, 2021).

2.3 Ataques Cibernéticos

A Cisco, grande do ramo de redes de computadores, define ataque cibernético como:

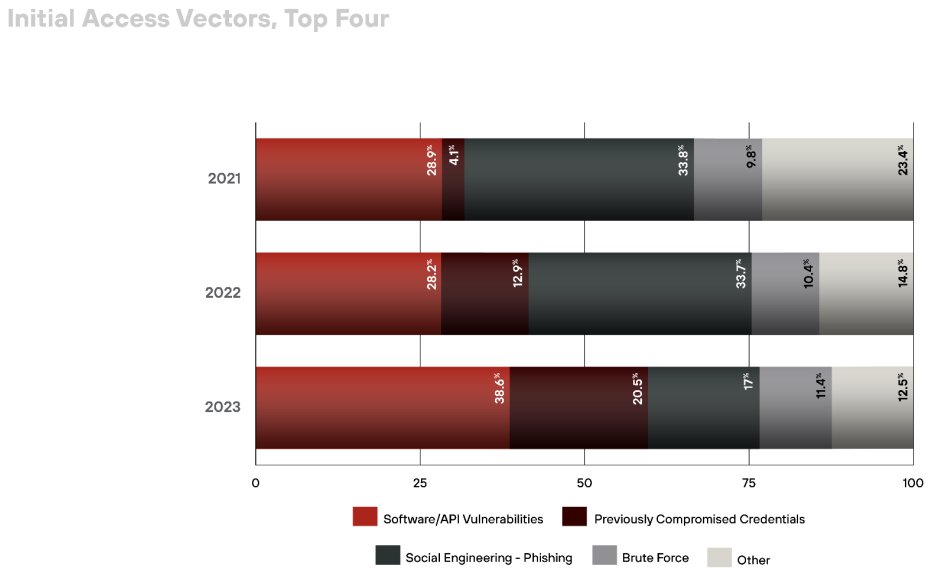
> Um ciberataque é uma tentativa maliciosa e deliberada de um indivíduo ou organização para violar o sistema de informações de outro indivíduo ou organização. Normalmente, o atacante busca algum tipo de benefício ao perturbar a rede da vítima. (CISCO, 2024)

Cibercriminosos procuram minimizar interações humanas durante a condução de ataques, ações de engenharia social ou ataques físicos podem levantar suspeitas caso haja algum deslize. Em vez disso, eles tendem a priorizar métodos que exploram vulnerabilidades técnicas para comprometer os sistemas das vítimas. De forma mais direta e, eventualmente, até automatizando o processo para possibilitar mais comprometimentos em um menor

tempo. O ataque cibernético é um procedimento para explorar a vulnerabilidade do sistema (PALOALTO, 2024).

Um estudo conduzido pela Unit 42, divisão de resposta a incidentes da empresa Palo Alto Networks, mostrou que os ataques realizados por notórios grupos de cibercriminosos, que afetam softwares e APIs, aumentaram de 28,2% para 38,6% durante o ano de 2023 (PALOALTO, 2024). O estudo, que mostra um problema latente na segurança dos usuários, é feito todo ano para comparação dos períodos passados, o gráfico como Figura 3 é apresentado.

Figura 3 – Técnicas de acesso inicial 2023.



Fonte: (PALOALTO, 2024)

Seguindo a mesma linha de raciocínio de se minimizar ataques com interação humana, o estudo revela que o abuso de credenciais vazadas também cresceu significativamente em relação aos anos anteriores. Isso indica uma tendência crescente entre os cibercriminosos de explorar métodos menos visíveis, mais discretos e automatizados para obter acesso a sistemas, evitando ações que possam levantar suspeitas e comprometer suas operações. (PALOALTO, 2024)

2.4 Ataques de Injeção

O ataque de injeção se trata de uma subcategoria da vulnerabilidade de entrada não verificada ou não higienizada (já mencionada seção de tipo de vulnerabilidades) (WEI; MUTHUPRASANNA; KOTHARI, 2006). Ocorre quando o usuário é capaz de mudar o fluxo de execução do código original, alterando a sintaxe com a inserção de instruções

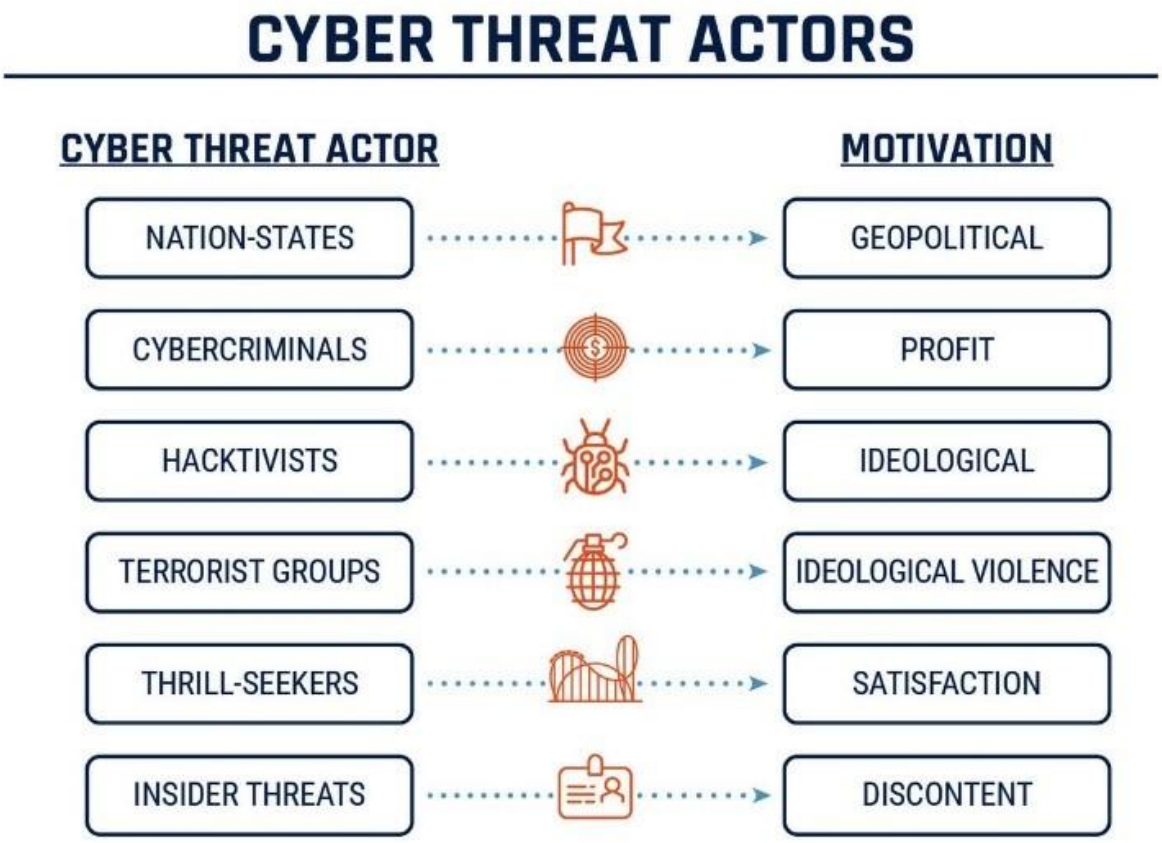
arbitrárias. Neste caso, entradas fornecidas pelo usuário são interpretadas como comandos ou consultas isoladas numa `string`. Os atacantes podem explorar essa vulnerabilidade inserindo comandos maliciosos nessas *strings*, que então são erroneamente executados como parte do código legítimo do sistema (MARIMUTHU, 2013).

2.4.1 Impactos

Ataques cibernéticos tendem a ocorrer por diversos motivos, mais comumente criminosos são motivados por ganhos financeiros. Roubo de informações para fins de extorsão, ou venda desses dados posteriormente. (IBM, 2024)

A Figura 4 (NICYBER, 2024) traz os nominais para os tipos de ameaças, ciber criminosos e suas motivações. Com isso é possível entender melhor quais tipos de ameaças uma aplicação está suscetível.

Figura 4 – Principais motivações por trás de crimes ciberéticos



Fonte: (NICYBER, 2024)

Os grupos conhecidos como **Nation-State** (ou state-sponsored actors) são patrocinados pelo estado em que atuam e são motivados por questões políticas. Já a ideia por trás de grupos **Hacktivistas** é causar danos como forma de se opor e protestar contra governos que promovem ideias autoritárias. Ambos os tipos de grupos visam impactar as

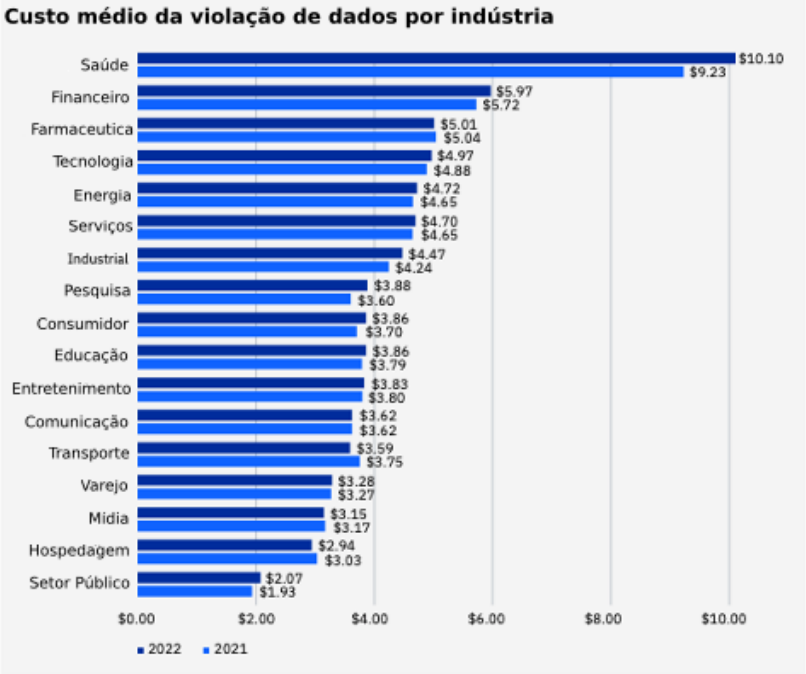
operações de governos e estados de oposição, causando geralmente a indisponibilidade dos serviços online desses estados ou roubando documentos confidenciais para abusos futuros, ou exposição (GUERRERO-SAADE, 2022).

O termo **cibercriminoso** é mais generalista e, assim como o conceito de **insider threat** (ou ameaça interna), refere-se a indivíduos que buscam ganhos pessoais através de suas ações maliciosas. No entanto, além desse ganho, os cibercriminosos sempre deixam um rastro de destruição. Quando o ataque é direcionado a uma empresa, isso pode afetar significativamente a confiança dos clientes na organização, além de trazer possíveis implicações legais (ECOTRUST, 2023).

A indústria da saúde é frequentemente alvo de ataques devido aos seus enormes bancos de dados contendo informações extremamente sensíveis, como dados de seguros de saúde, números de registros médicos e dados pessoais que podem ser vendidos ou usados para extorsão. Além disso, a área movimenta grandes valores, sendo essencial para todos. A falta de planos de contenção desenvolvidos em hospitais e clínicas torna-os alvos mais fáceis e procurados pelos atacantes (RESEARCH, 2023). Cibercriminosos também devem aproveitar os dados coletados para ataques de engenharia social, já que acabam adquirindo informações valiosas sobre os alvos.

Conforme exposto pela empresa de segurança Checkpoint (RESEARCH, 2023), após pesquisas, as organizações de saúde em todo o mundo, sofreram uma média de 1.463 ataques cibernéticos por semana em 2022, um aumento de 74% em comparação com 2021. Estimas-se um total de 422 milhões de vítimas impactadas (ITRC, 2023a).

Figura 5 – Custo médio do comprometimento por área de atuação.



Fonte: Adaptado de (HEALTH, 2024)

Na área da saúde, custo médio de cada violação é de cerca de US 10 milhões, contando com as implicações legais e medidas de contenção para novos ataques (HEALTH, 2024; ITRC, 2023a). Como mostrado na Figura 5, o valor teve um salto desde o ano anterior quando comparado com outras áreas, como ramo da tecnologia ou distribuição de energia.

Voltando à falha de XSS, como já mencionado, a Tabela 1 mostra alguns ataques que envolveram exploração dessa vulnerabilidade e seu impacto.

Alvo	**Descrição**	**Ano**	**Impacto**
British Airways	Falhas de XSS afetando uma dependência utilizada no website foi explorada para redirecionar usuário a uma página falsa.	2018	Centena de milhares de dólares em prejuízo com transações feitas para a página fraudulenta (MORADOV, 2022).
eBay	Um parâmetro vulnerável que recebia URL para redirecionamento não era validado. Cibercriminosos abusaram da falha para executar códigos e roubar sessão (MORADOV, 2022).	2015	O roubo de sessão permitiu acesso total à conta de vendedores, o que permitiu roubo de dados críticos.
UK Parliament Web Site	Falhas XSS foram encontradas no mecanismo de busca do site do Parlamento do Reino Unido, permitindo que os usuários escrevessem código para acessar imagens, vídeos e solicitar senhas (GUPTA, 2015).	2014	O principal impacto foi a desinformação, pois era possível alterar o conteúdo da página (BOTEZATU, 2013).
Yahoo Web Site	Vitimas receberam e-mails com link para um site malicioso, que explorava a falha em um subdomínio do Yahoo para roubar os cookies de sessão do usuário no webmail.	2013	Cibercriminosos tiveram acesso à conta do usuário no webmail, que lidou com a exposição de dados críticos.

Tabela 1 – Grandes ataques XSS e seus impactos

Com base na Tabela 1, é possível perceber que o objetivo e principal impacto da prática é o roubo da conta do usuário do site, assim tomando ações em seu nome. Caso o usuário comprometido tenha permissão de administrador, o atacante consegue facilmente tomar conta de toda a aplicação, e impactar negativamente seus usuários.

O objetivo final da segurança de aplicações é reduzir o tempo de inatividade e aumentar a confiança dos usuários nos serviços prestados. Isso envolve a implementação de medidas robustas para proteger contra ameaças, garantir a integridade dos dados e assegurar que as aplicações funcionem de maneira contínua e eficiente. Ao fazer isso, as empresas podem proporcionar uma experiência mais confiável e segura para seus usuários, aumentando o índice de adesão ao serviço (CLOUDFLARE, 2022).

2.4.2 Mitigações contra ataques de Injeção

As estratégias para mitigar a exploração de vulnerabilidades que permitem a injeção de código, podem ser divididas em dois modelos principais. O primeiro envolve a higienização de entradas utilizando funções de *escaping* nativas da linguagem de programação em uso, como com a utilização de procedimentos (stored procedures) (WEI; MUTHUPRA-

SANNA; KOTHARI, 2006), ou funções como `mysql_real_escape_string`, provida pela linguagem PHP (Medeiros, Iberia; Beatriz, Miguel; Neves, Nuno; Correia, Miguel, 2017), ambos exemplos quando se tratando especificamente de questões relacionadas à injeção de SQL (ITRC, 2023b). A segunda opção inclui a utilização de métodos alternativos de higienização, como o emprego de bibliotecas de terceiros, ou sistemas mais robustos, como Firewalls de Aplicação Web (WAFs). Essas abordagens ajudam a proteger sistemas contra a inserção de código malicioso nas entradas de dados expostas ao usuário (Medeiros, Iberia; Beatriz, Miguel; Neves, Nuno; Correia, Miguel, 2017).

A higienização de dados é o processo de manter os dados limpos e, em conformidade com as políticas estabelecidas. No contexto de uma aplicação, isso significa garantir que os dados fornecidos pelo usuário estejam no formato desejado e não contenham informações que possam ser prejudiciais à aplicação (SYNC, 2024).

Problemas como a injeção de SQL possuem mitigações bem encaminhadas. A vulnerabilidade ocorre quando não se processa o comando de SQL e a entrada do usuário de forma separada. Uma instrução SQL é vulnerável a ter a lógica da instrução alterada pela entrada em tempo de execução, quando a aplicação envia a estrutura e a entrada da instrução juntas em uma única solicitação ao banco de dados. A forma mais comum de se impedir a exploração é com a utilização de *Prepared Statements* (a tradução seria, instruções preparadas) (M.E, 2016; THOMAS, 2009).

Nesse método, a estrutura do comando SQL é definida e compilada antes de ser utilizada, e a entrada do usuário é posteriormente passada como argumento para o comando já definido. Devido ao processamento ocorrer em duas etapas distintas, torna-se inviável explorar a entrada do usuário para realizar uma injeção de SQL. Como o comando SQL é preparado previamente com espaços reservados para os dados do usuário, qualquer tentativa de injeção será tratada como simples dados, sem afetar a lógica ou a estrutura da consulta. Isso oferece uma camada robusta de proteção contra ataques de injeção de SQL. (THOMAS, 2009).

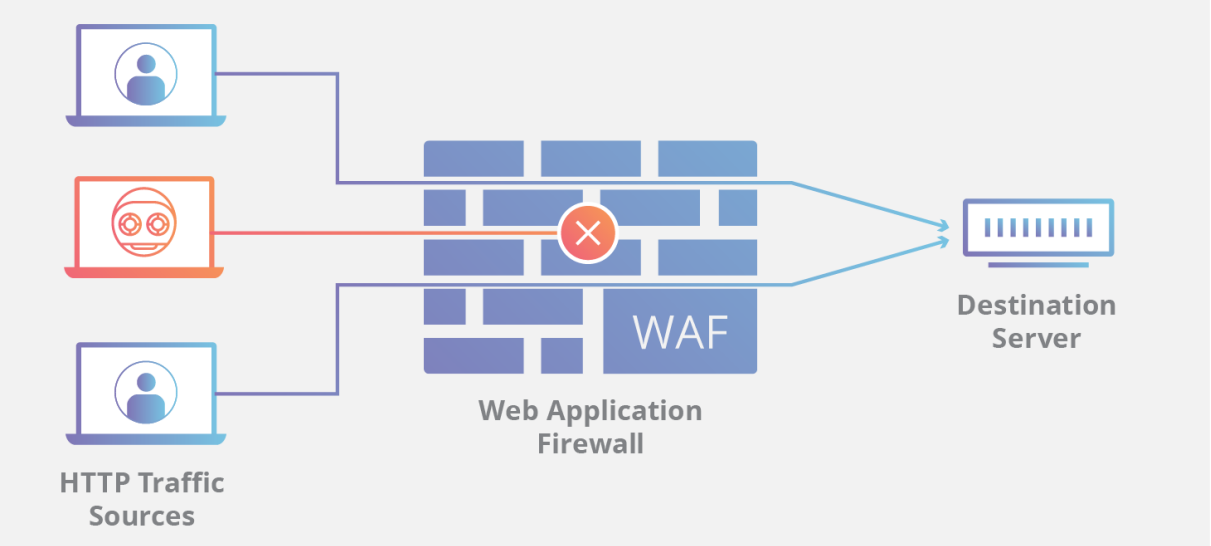
No entanto, outros ataques de injeção exigirão soluções mais dinâmicas.

2.4.3 *Firewall* de Aplicação

Firewalls de aplicação, ou *Web Application Firewalls* (WAF), são *firewalls* que operam na camada de aplicação. Esse tipo de *firewall* utiliza diversas políticas para monitorar e filtrar o tráfego da web que se dirige à aplicação, assegurando que o tráfego corresponde ao tipo esperado e permitido pela aplicação. Os WAFs verificam se as requisições estão em conformidade com o protocolo utilizado (HTTP no caso de aplicações web) e se não contêm conteúdos maliciosos. Dessa forma, protegem as aplicações contra uma ampla gama de ameaças, como ataques de injeção de SQL, *cross-site scripting* (XSS) e outros tipos

de problemas que podem comprometer a segurança e o bom funcionamento do sistema (MCHENRY, 2018).

Figura 6 – Ilustração, firewall de Aplicação



Fonte: (CLOUDFLARE, 2018)

As políticas do WAF visam proteger contra diversos tipos de ataques, especialmente injeções de código. Para isso, o *firewall* funciona baseado em proxy, sendo posicionado entre o cliente (usuário) e a aplicação. Dessa maneira, o usuário não tem acesso direto à aplicação.

Quando o usuário acessa a aplicação web, ele se conecta primeiramente ao WAF, um servidor intermediário, o proxy. O WAF inspeciona o tráfego, analisando os dados da requisição para identificar possíveis ameaças ou anomalias. Após essa verificação, o WAF encaminha os dados da requisição para a aplicação. A aplicação processa a requisição e envia a resposta de volta ao WAF, que então retorna os dados ao usuário.

Com essa abordagem, o firewall de aplicação tem acesso completo ao tráfego entre o usuário e a aplicação. Isso permite que o WAF inspecione cada byte dos dados trafegados, garantindo que estejam de acordo com os padrões de segurança estabelecidos e que não contenham conteúdos maliciosos (CLOUDFLARE, 2018). A Figura 6 ilustra a comunicação passando pelo WAF.

Existem vários tipos de WAFs, alguns são mais indicados para determinados cenários. O que determina o uso de um tipo de WAF é como e com qual eficiência se resolve o problema do bloqueio de acessos prejudiciais.

2.4.3.1 White-Listing

Estudos avaliam a aplicação de *white-list* essa aproximação menciona que a forma de tratamento contra injeção é, na verdade, muito mais simples. Basta apenas garantir que

o formato dos dados de entrada ou do conteúdo dos parâmetros a serem utilizados, estejam na forma coerente ao formato desejado. O principal problema desse tipo de WAF é a exigência de plena noção do funcionamento da aplicação, ter mapeado todas as entradas de dados e os tipos ou formatos de dados trafegados. Essa aproximação utilizará normalmente de expressões regulares para construir o padrão que o WAF deve permitir (GUNAWAN, 2018).

2.4.3.2 Uso de correspondência de padrões

Correspondência de padrões (ou *pattern matching*) é um termo mais generalista e existem várias formas de se aplicar. Mas *pattern matching* se baseia em uma lista de políticas, padrões pré estabelecidos que o acesso deve se adequar para passar. A diferença de *white-listing*, é que esse conjunto de técnicas especificam padrões maliciosos, caso o tráfego corresponda algum padrão já definido, conhecidamente malicioso, ele é rejeitado. Essa aproximação procura pela anomalia, e não valida se o dado está correto.

Várias aplicações do método estão em uso para a proteção de websites, algumas soluções de código aberto podem ser implementadas na aplicação web de forma gratuita, soluções comerciais exigem o uso de redes de conteúdo distribuídas (CDN) (FAROOQ, 2013). No entanto, soluções comerciais, como Cloudflare (CLOUDFLARE, 2024), Imperva (IMPERVA, 2024) e Akamai (AKAMAI, 2024) são mais caras por oferecerem muitas outras funções além do *firewall* de aplicação, muitas vezes desnecessárias caso apenas proteção seja o objetivo.

Soluções comerciais, com proteção de borda, vão oferecer algumas outras capacidades interessantes, que soluções de código aberto terão mais dificuldade em suprir, como a capacidade de detectar ataques de força bruta e negação de serviço por estresse (FAROOQ, 2013).

Um dos WAF de código aberto mais utilizados é o ModSecurity. O objetivo da ferramenta é proteger aplicações contra ataques de injeção, além de bloquear requisição contendo anomalias possivelmente prejudiciais. A ferramenta oferece funcionalidades para o registro de atividades, monitoramento e controle de acesso. Todas as requisições tidas como maliciosas e bloqueadas são registradas, contendo detalhes do tipo de ataque detectado. Com isso, os dados podem ser auditados para o entendimento da cadeia de ataque e definição de estratégias para mitigação de possíveis vulnerabilidades (MODSECURITY, 2024; RIZKI et al., 2020).

As ações do ModSecurity são tomadas com base nas regras, sendo suas políticas baseadas em expressões regulares. A seguir há um exemplo de regra para detectar ataques de injeção de SQL.

```
SecRule REQUEST_COOKIES|!REQUEST_COOKIES:
```

```
  /__utm/|REQUEST_COOKIES_NAMES|ARGS_NAMES|ARGS|XML:
  /* "@rx (?i)union.*?select.*?from" \
"id:942270,\
phase:2,\
block,\
capture,\
t:none,t:urlDecodeUni,\
msg:'Looking for basic sql injection.
  Common attack string for mysql, oracle and others',\
logdata:'Matched Data: %{TX.0}
  found within %{MATCHED_VAR_NAME}: %{MATCHED_VAR}',\
tag:'language-multi',\
tag:'attack-sqli',\
ver:'OWASP_CRS/4.4.0-dev',\
severity:'CRITICAL',\
setvar:'tx.sql_injection_score=+%{tx.critical_anomaly_score}',\
setvar:'tx.inbound_anomaly_score_pl1=+%{tx.critical_anomaly_score}'"
```

A regra do ModSecurity acima é projetada para detectar e bloquear tentativas básicas de injeção de SQL. Ela irá avaliar os argumentos da requisição por padrões maliciosos: cookies, nomes e valores de parâmetros da URL, e elementos XML na requisição (como especificado na primeira linha). É utilizada uma expressão regular que procura pelas palavras-chave *union*, *select* e *from*. A regra possui tags e variáveis de pontuação, usadas para categorização e avaliação de severidade dessa detecção, com uma mensagem descritiva detalhando a natureza da ameaça. Normalmente, há várias regras com pontuações diferentes que, se uma requisição for detectada por regras, a pontuação de todas essas regras são somadas, e, caso atinja um determinado valor (10 ponto normalmente), a requisição é bloqueada. Dessa forma, cada detecção isolada representa uma anomalia, não uma conclusão definitivamente, mas um indicador parcial. Diferentes combinações de regras podem frequentemente ser observadas ocorrendo juntas durante diferentes tipos de ataques. Esse modelo é conhecido como *anomaly-scoring*, que necessita de múltiplas regras para atribuir alguma criticidade ao acesso (MODSECURITY, 2024; AKAMAI, 2015).

```
http://domain.com/blog?article=carros'UNION SELECT user FROM from users;
```

Caso a requisição contenha uma injeção de comando SQL, no formato como visto acima, que tenta abusar da vulnerabilidade para extrair dados de usuários da tabela `users`, o acesso seria devidamente bloqueado, pois o padrão possui alguma das palavras-chave especificadas na política do ModSecurity, como `UNION`, `SELECT` e `FROM`.

Mod Security é executado no servidor web, o que é uma carga adicional no servidor que, dependendo do volume de tráfego e a preparação do servidor para lidar com isso, irá impactar negativamente o desempenho e diminui a capacidade de atender os usuários. Além disso, a criação de regras ou políticas, é uma tarefa bem complexa, que exige estudos e testes bem abrangentes para diminuir erros (FAROOQ, 2013; MODSECURITY, 2024).

A desvantagem dessa tática, é que ela se torna ineficaz contra novos tipos de ataques, pois não haverá assinaturas disponíveis para detectá-los. Os *firewalls* de aplicação que utilizam correspondência de padrões dependem de assinaturas predefinidas para identificar ameaças e ataques desconhecidos ou recém-criados podem passar despercebidos até que novas assinaturas sejam desenvolvidas, bem testadas e implementadas (MARIANNE, 2020).

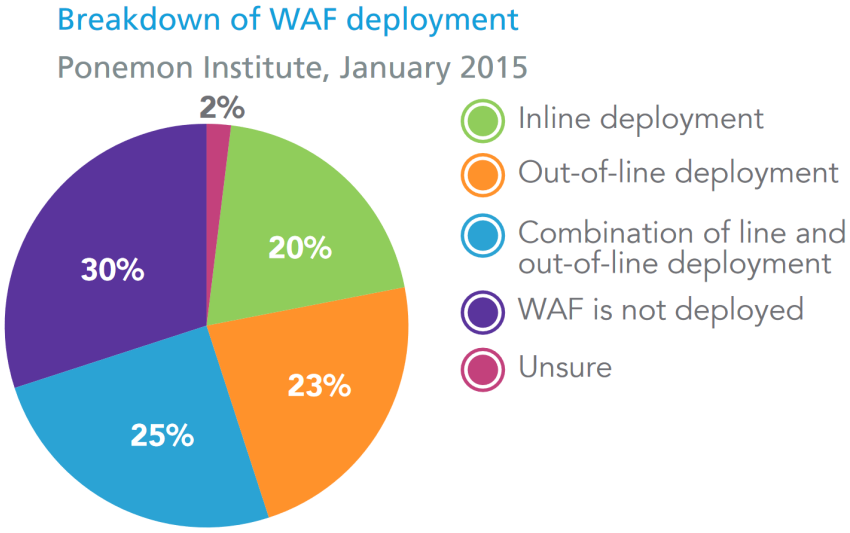
2.5 Falso-Positivo e Falso-Negativo

Um ponto importante no gerenciamento de um *firewall* de aplicação baseado em correspondência de padrões, é a taxa de falso-positivo e falso-negativo que o WAF gera (falso-positivo é discutido em uma seção própria). A taxa de falso-positivo se trata da quantidade de tráfego legítimo, sem propriedades prejudiciais, que o WAF bloqueia como se fosse algo malicioso. Os falsos-negativos ocorrem quando as solicitações maliciosas, com um potencial prejudicial, não são bloqueadas (IBM, 2020).

Analistas de segurança tendem a considerar o uso de WAFs como algo complexo de se implementar. O WAF se baseia em uma série de políticas, assinaturas de ataques já conhecidos. No entanto, novos vetores de ataque, ou permutações dessas assinaturas surgem a todo momento, muitas vezes tornando a política defasada. Outro ponto é que, características de tráfego legítimo mudam de aplicação para aplicação, muitas vezes sendo necessário ajustes adicionais. Com base nisso, se torna um desafio compreender ou quantificar a eficácia da solução de WAF de um ambiente, em tempo real e especialmente em termos de precisão, desempenho e sobrecarga gerada no sistema (AKAMAI, 2015).

Um estudo feito pela Ponemon sobre estratégias de uso de WAF, expôs os desafios enfrentados na indústria de segurança ao utilizar a ferramenta. A pesquisa mostrou que, de organizações que possuem serviço de firewall de aplicações, 23% utilizam a ferramenta em um modo que apenas registra eventos de segurança, que não aplica bloqueios, enquanto 20% utilizam realmente a solução para proteção da aplicação web, aplicando bloqueio no tráfego malicioso identificado (AKAMAI, 2015).

Figura 7 – Response to the question, "What best describes your organization's approach to WAF?"



Fonte: (AKAMAI, 2015)

Dos 594 profissionais de TI, que cuidam de segurança de aplicações, a Figura 7 mostra que 48% ou não usam a solução, ou usam parcialmente. Os profissionais entrevistados, que se encontram dentro dessa porcentagem, alegam alguns fatores que motivam a desabilitação da ferramenta. Foi mencionado que a solução traz consigo degradação no desempenho da aplicação ou alta complexidade no gerenciamento, balançamento entre falso-positivo e falso-negativo. As organizações adquirem a ferramenta visando implementar para bloqueio (inline), mas desativam quando percebem impactos em usuários legítimos (AKAMAI, 2015).

Para entender como se mensura a acurácia, existem mais dois fatores importantes: o verdadeiro positivo, que se trata dos ataques devidamente identificados e bloqueados; e o verdadeiro negativo, acessos legítimos não bloqueados.

No estudo sobre firewall de aplicação (OUISSEM, 2019), no qual é proposto uma nova modalidade de WAF baseado em correspondência de padrões, foram feitos testes com o ModSecurity utilizando o banco de políticas padrão de código aberto, Core Rulesets (4, 2024), na configuração padrão da solução.

ModSecurity	Verdade		Predição Geral	Precisão
	Ataque	Legítimo		
Ataque	TP: 179	FP: 8	187	95.72%
Legítimo	FN: 39	TN: 14	53	26.42%
Verdade Total	218	22	240	
Recall	82.11%	63.64%		

Tabela 2 – Performace do ModSecurity

Na Tabela 2 de confusão, a qual mostra a tabela de confusão de testes feitos com o ModSecurity, é possível ver que, o ModSecurity classificou corretamente 179 acessos maliciosos e 14 acessos legítimos. Porém, a ferramenta classificou de forma equivocada 39 acessos, que eram ataques, mas passaram despercebidos e 8 acessos legítimos forma classificados como ataques (OUISSEM, 2019). A acurácia geral foi de 80.4%, o que mostra que a ferramenta ainda teria que ser ajustada ao cenário para atingir melhores níveis.

A acurácia geral pode ser mensurada a partir da seguinte equação (como mostra feito em (OUISSEM, 2019)), onde P é a somatória de todos os positivos e N a somatória de todos os negativos:

$$\frac{TP \ + \ TN}{P \ + \ N} = Acurácia\ Geral$$

Para os testes, foi utilizada uma ferramenta automatizada para avaliação de WAF, chamada WebSecBench (WEBSECBENCH, 2019), ela se baseia nos vetores de ataque da OWASP Top 10 (OWASP..., 2021) para a produção do tráfego malicioso, e tráfego propenso a gerar falso-positivo.

Com a aplicação desse estudo, é possível mensurar o quanto a solução está adequada à aplicação, e pode ser utilizada em produção (AKAMAI, 2015).

2.6 Análise Gramatical

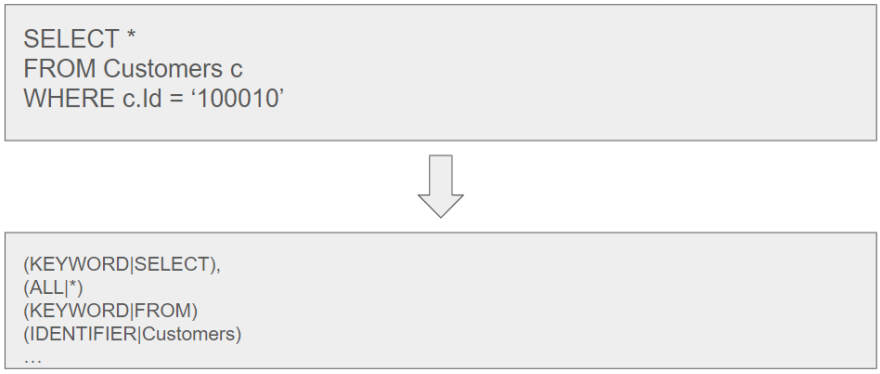
Toda linguagem de programação deve possuir uma série de regras, para a descrição de como se estrutura um programa. A sintaxe de uma linguagem pode ser descrita por uma gramática livre de contexto, e para isso é comum que se utilize a notação BNF (Backus-Naur Form), que descreve recursivamente a combinatória de tokens possíveis. Para ser possível analisar uma série de dígitos e verificar se pertence a uma linguagem, há alguns passos que devem ser avaliados (ULLMAN, 1995).

2.6.1 Análise Léxica

O analisador léxico é um algoritmo que processa uma sequência de caracteres e a divide em tokens, sendo unidades significativas da linguagem, como palavras-chave, identificadores e operadores. O token então é a menor unidade de uma linguagem, sendo uma palavra (uma sequência de caracteres) ou símbolo. Sozinho o token não significa nada, apenas em um contexto ele pode significar alguma ação. Além de segmentar a sequência de dados em tokens, ele verifica se há sequências de dados que não pertencem à linguagem, identificando possíveis erros léxicos e alertando caso esse cenário ocorra. Cada token é classificado em uma categoria específica, como identificador ou operador, facilitando a análise sintática subsequente. Dessa forma, o analisador léxico transforma a entrada em

um formato estruturado e compreensível para as etapas seguintes da análise simática (DELAMARO, 2014).

Figura 8 – Decomposição da expressão em tokens



Fonte: (DESCARTES, 2024)

Na Figura 8 há um exemplo de como se pareceria uma separação dos tokens de uma consulta SQL. Inicialmente existe uma sequência de caracteres que compõem uma consulta, mas essa informação não serve, o lexer irá analisar essa sequência procurando por sequências menores, os tokens, que podem significar algo para a gramática em questão. Normalmente os tokens, comumente chamados de lexemas (o token como aparece no código inicial), são divididos por espaços. Então, deve ser possível quebrar essa sequência quando aparecer um espaço e analisar se a sequência menor significa algo, um lexema. Cada lexema pode ser categorizado de forma diferente, o `SELECT` é classificado como `KEYWORD`, e o `Customers` é um `IDENTIFIER` (DESCARTES, 2024).

2.6.2 Análise Sintática

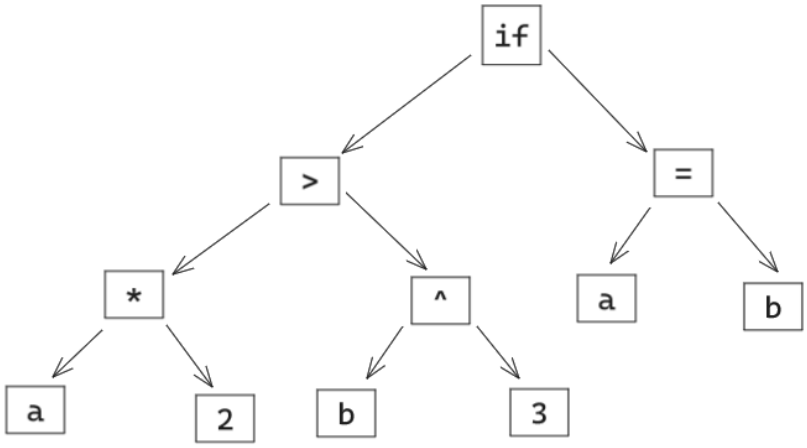
Na análise sintática, é verificada a coesão da sequência de tokens contido no código-fonte. Nessa etapa é verificado se essa sequência compõe um programa elegível para interpretação (ou compilação), ou se há erros de sintaxe (TAMAE, 2008). O analisador sintático recebe a sequência de tokens retornados do analisador léxico, então verifica-se se essa sequência de tokens poderia ser gerada pela gramática, ou não gerando assim erros de sintaxe. Durante a análise sintática algumas tarefas podem ser conduzidas, como a coleta de informações sobre os tokens, classificação dos tipos de dados com base em uma tabela de definições (ULLMAN, 1995).

Por exemplo, considere o seguinte trecho de código:

```
if (a * 2 > b ^ 3)
    a = b
```

O analisador sintático deve reconhecer o token com base em uma série de regras. A palavra reservada `if` sempre deve existir ao lado de uma abertura de parênteses `(`. Dentro dessa abertura deve existir uma expressão como `a > b`, seguida de um fechamento de parênteses `)`. Caso alguma regra não seja satisfeita, esse programa deve ser rejeitado. O analisador sintático desempenha seu papel enquanto constrói uma árvore sintática da estrutura. Para trecho de código acima, a árvore seria semelhante a qual é mostrada na Figura 9 (DELAMARO, 2014).

Figura 9 – Exemplo de árvore sintática.



Fonte: Elaborado pelo autor.

2.6.3 Análise Semântica

A análise semântica é a terceira etapa do processo de verificação de um programa, sucedendo à análise léxica e a análise sintática. Nesta fase, é verificado se os aspectos semânticos das construções do código estão corretos, ou seja, se há coerência em relação ao significado e às regras semânticas da linguagem. O analisador semântico verifica, por exemplo, se as variáveis foram declaradas antes de serem usadas, se os tipos de dados nas operações são compatíveis, e se as funções são chamadas com o número e tipo corretos de argumentos. Além disso, ele assegura que os identificadores estão sendo utilizados no escopo adequado que as operações realizadas são semanticamente válidas. Portanto, a análise semântica garante que o código não apenas tenha uma estrutura correta, mas também que faça sentido lógico no contexto da linguagem de programação (DELAMARO, 2014).

Por exemplo, `if (a > b)` e `a` for uma **string**, essa construção não faz sentido, então um erro deve surgir desse trecho.

Como base para a análise, a árvore sintática criada é utilizada. Além disso, ele dependerá de uma tabela de símbolos, onde variáveis e nomes de função já declaradas são

armazenadas (DELAMARO, 2014; TAMAE, 2008).

Após essas três etapas, análise léxica, sintática e semântica, é possível verificar se determinada sequência pode ter sido gerada por uma determinada gramática, e com isso validar se ela existe (ULLMAN, 1995).

3 Desenvolvimento

Este capítulo destina-se a descrição detalhada das etapas para a implementação do projeto, necessidades e possíveis requisitos.

O objetivo do trabalho foi a proposta e o desenvolvimento de uma prova de conceito para um *firewall* de aplicação baseado em análise sintática, visando promover uma maior acertabilidade na detecção de ataques baseados em injeção de linguagem SQL principalmente em ambiente web, tornando em um processo determinístico a identificação de abusos.

Este capítulo apresenta como foi o processo de implementação dessa solução, para atingir o objetivo do trabalho.

3.1 Escolha Da Gramática

A gramática escolhida para a implementação se tratou de um subgrupo de SQL, uma versão simplificada, porém, completa o suficiente para possibilitar a criação de uma prova de conceito robusta de um *firewall* de aplicações.

SQL continua sendo uma linguagem amplamente explorada em ataques cibernéticos, especialmente contra sistemas legados. Esses sistemas geralmente não recebem atualizações frequentes, pois há o risco de que mudanças causem erros e mau funcionamento. Como resultado, muitos sistemas críticos permanecem vulneráveis e frequentemente, quando expostos, se tornam alvos de ataques.

Além disso, a implementação de uma variação simplificada de SQL é viável dado a estrutura da linguagem.

3.1.1 Escolha Dos Comandos Disponíveis

Para uma prova de conceito de um WAF baseado em análise sintática, não seria necessário implementar todos os tokens e comandos definidos na gramática do SQL, então foram escolhidos os comandos mais adequados para serem incluídos no fluxo de análise do analisador, com base em alguns fatores.

Grande parte dos ataques que exploram injeção de SQL pretende seguir com a extração dos dados críticos armazenados no banco de dados do site vulnerável. Com base nisso, foram implementados apenas os comandos que possibilitam esse tipo de interação. Portanto, foram implementados apenas comandos que permitem a consulta de dados como `SELECT` e seus argumentos, além da possibilidade de se encadear consultas com o comando

`UNION`. O comando `SELECT` permite uma série de subcomandos para o ajuste e filtragem de seu retorno, mas apenas alguns foram implementados, como `FROM`, `WHERE` e `LIMIT`.

3.2 Escolha Do Tipo de Parser

Para a técnica de análise sintática, foi escolhida a utilização de um analisador recursivo descendente, devido à sua simplicidade e facilidade de implementação. Com esse tipo de analisador, cada regra da gramática pode ser diretamente mapeada para uma função, criando-se assim um código bem claro e legível. Isso facilita o crescimento do código em adições futuras, caso necessário.

Além disso, o analisador recursivo descendente permite um controle detalhado sobre o fluxo de *parsing*, possibilitando a descoberta e tratamento de erros de forma precisa pela fácil legibilidade das sequências de análise. Essa abordagem é útil para SQL, que possui várias particularidades na gramática. Como o SQL utilizado é um subgrupo com uma estrutura relativamente simples, a análise *top-down* definida no analisador recursivo descendente é suficiente, sem a necessidade de algoritmos mais complexos.

3.3 Arquitetura

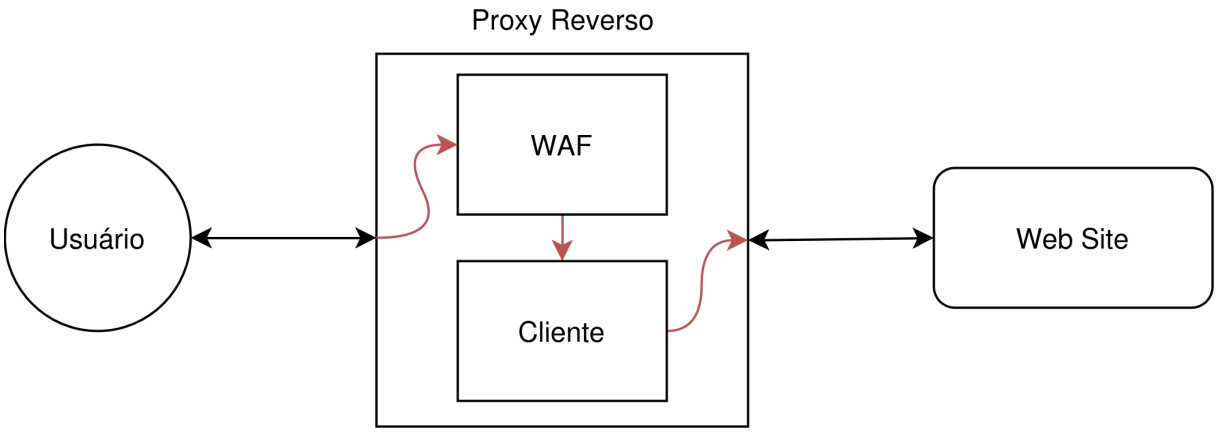
O objetivo final foi desenvolver uma ferramenta capaz de analisar o tráfego HTTP direcionado a uma aplicação vulnerável e identificar comandos potencialmente prejudiciais a essa aplicação web. Para que a ferramenta pudesse examinar os dados antes de serem processados pela aplicação, duas abordagens foram consideradas: implementar a ferramenta diretamente na aplicação web, integrando-a ao seu processo de análise do dado, ou implementá-la em um *proxy* reverso, que interceptaria o tráfego antes de ele chegar à aplicação, permitindo a filtragem e avaliação de riscos de forma independente.

Caso fosse construída uma biblioteca implementável, esse processo poderia dificultar o uso em aplicações que não foram construídas na linguagem usada para desenvolver a solução, o que adicionaria uma dificuldade a mais no uso e testes da implementação, a reescrita da solução.

A forma escolhida foi a implementação da solução em um *proxy* reverso, que tornaria o funcionamento do *firewall* de aplicação baseado em análise sintática totalmente independente da aplicação em si. Isso facilita o teste em diversos ambientes e não exige o gerenciamento da aplicação para os testes e verificações, o que possibilita inclusive o uso da solução em aplicações de terceiros.

Conforme mostrado na Figura 10, o usuário irá acessar o proxy reverso ao invés da aplicação web, os dados da requisição serão processados pelo WAF, o analisador sintático. Caso a requisição esteja em conformidade, ou seja, não tenha dados maliciosos, ela será

Figura 10 – Arquitetura do proxy reverso



Fonte: Elaborado pelo autor

direcionada ao *upstream* (a aplicação web sendo protegida pela solução), que retornará o conteúdo ao *proxy* reverso, que por sua vez, entrega-o ao usuário.

O *proxy* reverso não deve causar impactos na experiência do usuário, tornar a comunicação mais lenta ou corromper dados pois isso causaria danos na usabilidade da aplicação.

Tanto o *proxy* reverso, quanto o *firewall* de aplicação foram desenvolvidos na mesma linguagem de programação, utilizando a mesma arquitetura de código, para que fossem amplamente compatíveis, e o *firewall* de aplicação acoplado ao *proxy* reverso como uma biblioteca, tornando-os uma solução única.

A configuração do *upstream*, a aplicação alvo que a solução protege deve ser dinâmica e de fácil modificação, para facilitar os testes.

3.4 Escolha Da Linguagem

Para desenvolver toda a solução, tanto o *proxy* reverso quanto o *firewall* de aplicação, a linguagem escolhida foi Golang, devido a sua sintaxe simples e de fácil compreensão. Além disso, Golang oferece uma vasta biblioteca padrão para desenvolvimento em diversos protocolos de comunicação, incluindo suporte robusto para HTTP, o que seria necessário no desenvolvimento do *proxy* reverso. Outro fator determinante foi a capacidade de compilar o código em um único binário, facilitando a distribuição e execução da solução sem necessidade de gerenciar dependências em cada máquina onde o sistema fosse implantado. Isso reduz a complexidade de configuração e garante um ambiente mais estável para testes e o uso da implementação.

3.5 Analisador

Como o analisador serve apenas para verificar se a sequência de caracteres está conforme a gramática define, e não tem a função de gerar código ou gerar um resultado com base na interpretação das sequências, a estrutura do analisador se torna simples.

3.5.1 O Lexer

O *Lexer* tem a função de separar os lexemas de uma sequência de caracteres. O *lexer* se trata de uma função que será executada pelo *parser*, e sempre quando executada, retorna o próximo token, o lexema e seu significado no contexto da gramática.

3.5.1.1 As estruturas

O *lexer* é definido o como uma estrutura chamada de `Lex`, que possui quatro atributos: `Code` se trata da sequência de entrada completa, o código SQL que será analisado; `Cursor`, a posição atual do `Code` sendo lida; `CChar` (*Current Character*) o caractere processado atualmente e `Ptr` (*pointer*) usado para auxiliar o `Cursor` na leitura de lexemas maiores do que um caractere.

Existe uma segunda estrutura chamada `Token`, que define o token da forma a ser utilizada pelo *parser*. A estrutura possui dois atributos, sendo eles `Type` que informa o tipo do lexema, e `Data` que possui o lexema.

3.5.1.2 Identificação da sequência

Ao projetar uma técnica para dividir uma sequência de caracteres em tokens, ou seja, fragmentos menores com significado específico na gramática, a abordagem mais imediata pode ser separar esses fragmentos pela ocorrência de espaços. Esse método é eficiente em vários cenários, mas para o SQL e o propósito em questão, ele apresenta algumas limitações. Isso se deve principalmente pelo fato de que podem existir comentários entre lexemas como no exemplo a seguir, então separar a sequência pelo espaço não é mais algo viável. (ISO, 2023)

```
SELECT/* Comentario */x/* Comentario */FROM/* Comentario */y;
```

Nesse caso, a separação por espaços não funciona, pois os comentários delimitam um token válido. Além disso, a presença de quebras de linha entre tokens exige um tratamento que vá além da simples divisão por espaços. O lexema pode estar ao lado de diversos símbolos que possuem algum significado para a gramática.

```
TOKEN;
TOKEN\x20
TOKEN/*
TOKEN#
```

```
5 TOKEN\n
6 TOKEN--
```

Para identificar corretamente os limites de um token, o *lexer* deve reconhecer uma quebra no padrão de uma sequência específica, ignorando comentários e espaços, para então identificar o início e o fim de cada lexema.

Considere a sequência de código SQL `SELECT x FROM y;`. Ao criar uma instância da estrutura `Lex`, o atributo `Code` armazena essa sequência, enquanto os outros atributos, como `Ptr` e `CChar`, começam com valor zero. Na primeira iteração, para descobrir o primeiro token, o *lexer* identifica o primeiro token, que corresponde à sequência de caracteres `SELECT`. Nesse ponto, `Ptr` ainda está em 0, e o cursor avança até alcançar o final do lexema que será identificado, após sua identificação, `Ptr` se iguala ao `Cursor`.

Para cada caractere `x` em `SELECT`, `x` é armazenado em `CChar` e verificado, caso ele seja uma letra ou um dígito, o `Cursor` recebe a posição do próximo caractere sendo o novo `x`, o qual é novamente armazenado em `CChar`, até que encontre um símbolo que não seja nem número dígito. Quando `x` é algo que não se trata nem de número, nem de dígito, a sequência antes desse `x`, desde o `Ptr` ao `Cursor`, é retornada como o token.

3.5.1.3 Definição do tipo

Após ter o lexema é necessário descobrir se ele possui algum significado na gramática, e qual é esse significado. A busca pelo significado é feita em uma estrutura de dicionário chave-valor, que possui o mapeamento de todos os possíveis lexemas implementados até então, e seus significados. Caso uma sequência de caracteres, como `AAAA` por exemplo seja buscada, essa sequência não possui um significado, então ela é definida como um **identificador**, que pode ter diversos significados no contexto da gramática, como ser uma variável, nome de função ou nome de tabela, ou até coluna. Caso haja o lexema pesquisado no dicionário, o valor correspondente é retornado, como o valor 1 para `SELECT` ou 3 para `FROM`.

```
1 tokensDisponiveis = {
2     "SELECT" : TipoSelect = 1
3     "UNION"  : TipoUnion  = 2
4     "FROM"   : TipoFrom   = 3
5     "LIMIT"  : TipoLimit  = 4
6     ...
7 }
```

Essa abordagem torna mais simples para adicionar novos lexemas ao *lexer* com base na evolução da ferramenta, pois é possível apenas adicionar a *string* correspondente ao lexema no dicionário e atribuir um novo valor único a ele.

O token descoberto é inserido em uma instância da estrutura `Token`, retornado para ser utilizado pelo *parser*.

```
Token {Data: SELECT, Type: 1}
Token {Data: x,      Type: 8}
Token {Data: FROM,   Type: 3}
Token {Data: y,      Type: 8}
Token {Data: ;,      Type: 9}
```

No exemplo acima 8 se trata de um **identificador**, que dependendo do contexto pode ser um nome de coluna, como o primeiro **identificador**, ou uma tabela no segundo **identificador**. Porém, o *lexer* apenas informa que se trata de um **identificador**.

3.5.1.4 Funções De Interface

As funções de interface interagem com o analisador, direta ou indiretamente.

O *lexer* é inicializado pela função `Init`, que recebe a expressão como argumento e retorna a estrutura `Lex` com os valores de seus atributos inicializados na forma padrão, `Code` contendo a expressão, sequência que será verificada pelo analisador. `Ptr` inicia-se como 0 e o `Cursor` também inicia-se como 0.

A função `GetToken` recebe a estrutura `Lex`, e para cada vez executada, retorna o próximo token como mencionado em 3.5.1.3.

3.5.1.5 Verificação de Comentário e Espaços

A função `GetToken` do *lexer* retorna os tokens encontrados na sequência, mas existem tokens que a função não retorna, como espaços em branco, quebras de linha, tabulações entre outros caracteres que não sejam úteis para a análise. Esses caracteres são apenas ignorados durante a verificação e o lexema. Após a ocorrência desses caracteres, não importa quantas ocorrências sejam, é retornado como token.

Comentários também podem ser ignorados pela implementação do *lexer*. Quando a função `GetToken` se depara com um bloco de comentário, definido na gramática do SQL, que se inicia um caractere barra seguido de uma estrela (exemplo do padrão `/*`), e termina com o mesmo padrão, porém de forma invertida (exemplo do padrão `*/`), o *lexer* ignora como um espaço em branco, não retornando como token. Nesse caso, o *lexer* irá retornar o próximo lexema válido como token, após o bloco de comentário. (ISO, 2023)

3.5.2 O Parser

O *parser* tem a função de analisar o texto, a sequência de entrada e verificar se está conforme as regras da gramática. Foi implementado um analisador recursivo descendente para verificar essa entrada de dados. Então, ele é usado apenas para verificar se a sequência

de entrada pertence ou não à gramática da linguagem SQL (seu subgrupo utilizado no projeto).

3.5.2.1 A Estrutura Parser

Para reter o estado do *parser*, foi criada uma estrutura chamada `Parser` que possui cinco atributos: `Lexer` se trata da estrutura descrita anteriormente no tópico sobre o *lexer*; `Token` se trata também da estrutura do token; `Error` é um valor *booleano* (seu estado pode ser apenas *true* ou *false*, verdadeiro ou falso), seu valor é positivo, *true*, quando um erro de sintaxe é descoberto, e `count` conta o tamanho da sequência congruente.

3.5.2.2 Funções De Interface

O *parser* começa na função denominada `Init` que recebe como argumento um novo *lexer* já inicializado. Essa função inicializa a estrutura `Parser` com os valores padrão, sendo `count` igual a 0, `Token` sendo nulo, e `Error` sendo falso.

A função `Eval` avalia a sequência e retorna dois valores ao final de sua execução, um inteiro que contém o resultado do atributo `count` da estrutura `Parser`, e um valor booleano, resultado do atributo `Error`. Então, a função `Eval` possibilita verificar se há um erro de sintaxe na de entrada, e quantos lexemas congruentes existem em sequência.

Para cada token a ser processado, a função do *Parser* `parserGetToken`, implementada usando a função `GetToken` do lexer, é executada atribuindo um novo token ao atributo `Token` da estrutura `Parser`. Então o parser faz as devidas checagens com base nesse atributo para seguir à próxima validação. A função `parserGetToken` também incrementa o valor de `count`, mostrando que um novo lexema da sequência de entrada foi verificada com sucesso.

Durante a análise da sequência de entrada, caso um erro de sintaxe seja descoberto, a análise é finalizada e retornada ao ponto da de entrada na função que iniciou recursão. O valor do atributo `Error` da estrutura `Parser` é configurada como **falso** e a função `Eval` do parser retorna informando o erro de sintaxe e a quantidade de tokens corretos na sequência de entrada, a expressão.

3.5.2.3 Análise da Expressão

Durante o desenvolvimento do trabalho foi decidido implementar apenas os comandos SQL principais, que permitissem o teste da prova de conceito. Esses comandos também foram limitados, para extrair funções menos importantes no olhar do teste.

A implementação foi feita pensando em tornar simples a adição de novas funcionalidades, sem necessitar de muitas modificações no código, além da adição das funções nos locais próprios.

A primeira função é a `SELECT`, para a consulta de dados. O bloco de **BNF** a seguir mostra como foi considerada a implementação do comando.

```
select_sttm ::= ( SELECT result_column ( ',' result_column ) *
    [ FROM table_or_subquery ( ',' table_or_subquery ) * ]
    [ WHERE expr ] [LIMIT expr]
```

A função `result_column` verifica o que é considerado uma coluna, que pode ser um identificador, uma expressão, um novo comando, ou uma cadeia de tudo isso separado por vírgula.

```
result_column ::= ( '*'
    | table_name '.' '*'
    | expr [ [ AS ] column_alias ] )
```

Uma `table_or_subquery` também pode ser resultado de várias operações, ou apenas um identificador com um determinado significado dentro da expressão ou então uma nova consulta encadeada. A seguinte **BNF** mostra o que foi considerado na formulação de tabela.

```
table_or_subquery ::= ( [ database_name '.' ] table_name
        [ [ AS ] table_alias ]
    | '(' ( table_or_subquery ( ',' table_or_subquery ) * ) ')'
    | '(' select_stmt ')' [ [ AS ] table_alias ] )
```

A função `expr` é o ponto de entrada para a análise de uma expressão na cláusula WHERE, ou em outros pontos. Ela lida com operadores *booleanos* (AND, OR) e verifica se, tanto o lado esquerdo quanto o lado direito de uma expressão estejam válidos. Podem existir expressões aninhadas, agrupadas entre parênteses. Isso permite que expressões como `(a = 1 OR b = (a = 1 AND b = 2))` sejam analisadas corretamente. Além do sinal de igual, sinais como $>$ e $<$ podem ser utilizados e devidamente verificados.

```
expr ::= ( literal_value | expr binary_operator expr )
```

A análise se restringe aos comandos implementados, mostrados acima, além do comando `UNION` que quando executado entra na função que lida com as definições de `SELECT`.

3.6 O Firewall De Aplicação

O *firewall* de aplicação (WAF) tem a função de receber uma *string*, extraída de algum parâmetro da requisição HTTP, tanto *GET* quanto *POST* (para a implementação aqui descrita, apenas parâmetros *GET* foram considerados), e verificar se existem padrões eventualmente prejudiciais para um vetor de interação, que possa estar vulnerável.

O problema enfrentado, nessa análise, foi descobrir como poderia ser feita a verificação de um comando que não estivesse completo, que existisse apenas uma parte desse comando, um fragmento da consulta SQL.

Por exemplo, o padrão `1 OR 1 = 1/*` por si só aparentemente não significa nada, porém, sendo enviado à aplicação, para em um campo vulnerável esse padrão seria encadeado na consulta e ter o seguinte resultado:

```
SELECT x FROM y WHERE z=1 OR 1 = 1/*;
```

Tendo em vista que, em muitas aplicações, utilizar o valor de um parâmetro HTTP é uma técnica para filtrar os resultados de uma consulta, posicionando diretamente o valor na cláusula `WHERE` é algo comum.

3.6.1 Técnica de Análise

A técnica inicialmente considerada envolvia visualizar o analisador recursivo descendente como uma árvore. A ideia era aplicar uma busca em largura nas ramificações da árvore para identificar onde a expressão seria analisada. Assim, a busca teria o objetivo encontrar o ponto da árvore onde se avalia uma sequência específica de tokens, como um token 1 seguido por um token `OR`, e depois outro token 1. Se houvesse um erro de sintaxe ao analisar uma determinada ramificação, isso indicaria que aquele ponto da árvore não era o local correto para começar a avaliação, e a busca prosseguiria para a próxima ramificação. A verificação de valores literais naturalmente ficariam nas pontas.

Essa aproximação para avaliar se os fragmentos de consulta estão coerentes seria complexo, pois o analisador deveria ser construído em volta dessa técnica de análise. A busca também adicionaria um tempo elevado para verificar as expressões de entrada, o que prejudicaria um ponto importante, uma solução que não adicione uma latência alta, tal que prejudicasse a experiência do usuário no uso da aplicação web.

3.6.1.1 Consulta Genérica

A segunda abordagem avaliada foi a definição de consultas genéricas, nas quais os valores dos parâmetros são posicionados antes da análise. Nessa abordagem, a consulta genérica teria uma estrutura semelhante as quais são normalmente utilizadas em aplicações web. Dessa forma, a sintaxe seria validada corretamente se os parâmetros seguissem o padrão esperado pela aplicação, assegurando uma análise eficiente sem comprometer a integridade da sintaxe da consulta.

```
SELECT x FROM y WHERE z=[VALOR];
```

Neste caso, o dado a ser avaliado seria inserido no local de `[VALOR]` da consulta, criando assim uma consulta válida.

3.6.2 Verificando Uma Manipulação Na Consulta

Quando o campo sendo verificado recebe um valor numérico, como uma identificação de produto que a aplicação retorna para o usuário ao consultar tal identificador, o tamanho da consulta SQL analisada pelo parser se mantém constante. A consulta genérica é concatenada com o valor do parâmetro, variando apenas o valor numérico específico inserido no parâmetro da requisição *GET*. Nesse contexto, se mais dados forem inseridos, resultando em tokens adicionais na sequência, essa diferença será detectável pela contagem final de tokens válidos em sequência, consulta genérica mais os tokens adicionais. Essa contagem é extraída pelo *parser* a partir do parâmetro `count`, permitindo a identificação de inserções não previstas na estrutura original da consulta.

Tendo em vista que a consulta `SELECT x FROM y WHERE z=1;` possui apenas 9 tokens, qualquer variação legítima dessa consulta na aplicação web também terá essa mesma quantidade de tokens, outras requisições que buscam o mesmo padrão de dados, com variações no valor, a consulta final terá o mesmo tamanho. Uma consulta que contenha padrões maliciosos como `SELECT x FROM y WHERE z=1 OR 1=1/*;` conterá mais tokens, 15 tokens nesse caso, então é possível assumir que há uma injeção de SQL quantidade de tokens superior à esperada.

No caso de se ter uma *string* sendo analisada, o resultado é o mesmo que se tivesse apenas um número inteiro. Em relação à contagem de tokens, uma *string* é vista como apenas um token e não vários em sequência para cada palavra. Então, o padrão `SELECT x FROM y WHERE z='qualquer dado';` tem apenas 9 tokens, então seria possível verificar novos tokens sendo inseridos para desviar o fluxo de execução da consulta SQL, encadeando novos comportamentos.

Outra forma possível para verificar a manipulação da consulta, seria pela verificação de erros de sintaxe. Tipicamente, se há algum token inválido na consulta ao banco de dados de uma aplicação web, isso irá gerar um erro de sintaxe. Inclusive, em muitos casos é possível identificar uma vulnerabilidade, um vetor de injeção de SQL pelo erro de sintaxe retornado ao usuário. A avaliação do WAF irá concatenar a consulta genérica com o valor do parâmetro *GET*, e então verificar com o analisador. Gerar erros de sintaxe não é um comportamento comum, então é possível concluir que a requisição se trata de algo possivelmente prejudicial à aplicação.

O padrão de consulta genérico é atualmente uma limitação, pois é possível verificar apenas vetores de número inteiro ou de *string*, o vetore que a consulta genérica em uso for compatível. Caso haja a tentativa de verificar uma *string* com uma consulta genérica de número inteiro, isso irá gerar erros devido à inexistência de aspas `SELECT x FROM y WHERE z=qualquer dado;`. Caso seja verificado um número inteiro com uma consulta genérica de *string* isso poderá gerar falso negativo em alguns padrões maliciosos que não contenham

aspas, como o padrão `SELECT x FROM y WHERE z="1 OR 1";` que há um acesso malicioso não efetivo, mas há uma tentativa.

3.6.2.1 Funções De Interface

O WAF pode ser utilizado como uma biblioteca com funções exportadas que permitem sua execução por outros sistemas, como o *proxy* reverso. Nesse caso, o *proxy* reverso integra o WAF em seu fluxo de análise da requisição, chamando suas funções para avaliar e inspecionar cada requisição antes que ela chegue à aplicação.

A função `New` retorna uma estrutura do tipo `Waf` sem parâmetros. A estrutura possui dois métodos que recebem uma `string` como parâmetro: `TestInt` utiliza de uma consulta genérica para avaliar números inteiros; `TestStr` que utiliza de uma consulta genérica para avaliar `strings`. O retorno de ambos métodos é valor *booleano* (falso ou verdadeiro). O retorno é falso caso injeção de SQL não tenha sido identificada na expressão processada, e verdadeiro caso haja injeção de SQL.

3.6.2.2 Verificação de Uma *Query String*

Como mencionado, a implementação do WAF se restringe apenas a verificação de dados nos parâmetros *GET* de uma requisição HTTP.

No momento que uma requisição chega ao *proxy* reverso, existe um algoritmo que fará uma iteração em todos os parâmetros da requisição HTTP. Para cada parâmetro encontrado, ele será processado pelo WAF. Caso se espere que o parâmetro receba um número inteiro, o método `TestInt` é utilizado, caso uma *string* o método `TestStr` é utilizado. O *proxy* reverso deve ter iniciado o Waf previamente com a função `New`, antes do processamento da requisição.

Caso o retorno da análise feita pelo WAF seja positivo, mais padrões de SQL tenham sido identificados na sequência de caracteres processada, proxy reverso retorna uma resposta HTTP com código de *status* 403.

4 Testes e Resultados

Para avaliar a utilidade do software, foi criada uma aplicação web vulnerável visando verificar se, ainda com o *firewall* de aplicação, é possível explorar a vulnerabilidade e extrair dados.

Os testes se basearam principalmente, na prática de *fuzzing*, o envio contínuo de requisições maliciosos para a aplicação alvo sendo protegida (MCNALLY et al., 2012), visando entender o padrão de requisições que não são bloqueadas quando, na verdade, deveriam ser. Com análise do *logs* de acesso do *proxy* reverso e a aplicação web, é possível ter a relação de requisições que passaram do *proxy* reverso para a aplicação, e assim validar a efetividade da solução.

Nessa seção serão apresentados os testes e resultados obtidos.

4.1 Aplicação Web Vulnerável

Como uma aplicação web funcional para testar a solução, foi criada uma API (Interface de programação de aplicações) simples que possui seu único parâmetro vulnerável à injeção de SQL.

A API foi construída também em Golang e possui apenas uma rota (a raíz `/`) que retorna dados de carros quando requisitado. O parâmetro *GET* vulnerável `id` permite especificar o número de identificação do carro sendo buscado. Esse parâmetro não verifica o tipo de dado e concatena diretamente à consulta SQL, então é possível encadear comandos de SQL para interagir com o banco de dados de forma arbitrária.

No exemplo a seguir foi utilizada a ferramenta `curl` de linha de comando para testar as respostas da API, que para os testes está em execução localmente e na porta TCP 8081. Em alguns testes, `curl` estará sendo utilizado por retornar a resposta apenas em texto no terminal, assim facilitando seu manuseio.

```
curl "http://0.0.0.0:8081/?id=1"

{"data":[
  {
    "brand": "Toyota",
    "model": "Corolla"
  }
]}
```

Quando a rota é acessada, uma linha de *log* é criada no terminal onde a aplicação está em execução, mostrando como foi feita a consulta no banco de dados que gerou tais

resultados.

```
2024/11/20 18:55:07 q='SELECT brand, model FROM cars WHERE id=1;'
```

Como um exemplo de injeção de SQL, é possível enviar o padrão `1 OR id=2` no parâmetro `id`, que naturalmente retornaria apenas um carro cadastrado, mas agora retorna dois, pelo fato de que existem os cadastros de identificação 1 e 2.

```
curl "http://0.0.0.0:8081/?id=1+OR+id=2"

{"data":[
  {
    "brand": "Toyota",
    "model": "Corolla"
  },
  {
    "brand": "Honda",
    "model": "Civic"
  }
]}
```

A seguinte linha de *log* emitida pela API mostra como foi executada a consulta no banco de dados.

```
2024/11/20 18:59:43 q='SELECT brand, model FROM cars WHERE id=1 OR id
    =2;'
```

Além da tabela que armazena os dados dos carros cadastrados, existe uma segunda tabela que armazena os dados dos donos desses carros, mas esses dados não devem ser acessíveis via API. Esses dados podem ser consultados e extraídos via interface web, apenas através da exploração da vulnerabilidade de que permite a injeção de SQL.

4.1.1 Teste Na API Com SQLMap

O SQLMap é uma das ferramentas mais amplamente utilizadas para auditoria e teste de vulnerabilidades relacionadas à injeção de SQL em interfaces web. Ele é projetado para identificar automaticamente falhas de segurança em aplicações vulneráveis, e permite que essas vulnerabilidades sejam exploradas de maneira controlada pela ferramenta. Além de detectar a existência de falhas, o SQLMap pode ser usado para explorar essas vulnerabilidades, possibilitando a extração de dados sensíveis do banco de dados por meio da aplicação comprometida. A robustez do SQLMap, junto da facilidade de uso, o tornam uma escolha popular entre profissionais de segurança para realizar testes de penetração e auditorias de sistemas. (STAMPAR, 2006)

A seguir, na Figura 11 há um exemplo de como SQLMap foi capaz descobrir a vulnerabilidade, e identificar os nomes das tabelas existentes no banco de dados da API.

Parte das informações emitidas pela ferramenta foram recortadas diminuir seu tamanho, mantendo apenas os dados mais importantes para o entendimento dos resultados.

As instruções de linha de comando utilizadas:

```
sqlmap -u "http://192.168.15.138:8081/?id=1" --dbms=sqlite --flush-
    session --tables
```

Figura 11 – Testando a API com sqlmap

```
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N]

sqlmap identified the following injection point(s) with a total of 17 HTTP(s) requests:
---
Parameter: id (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: id=1 AND 3729=3729

    Type: UNION query
    Title: Generic UNION query (NULL) - 2 columns
    Payload: id=1 UNION ALL SELECT CHAR(113,98,107,118,113)||CHAR(73,97,84,80,79,113,115,82,106,103,122,79,115,116,100,
122,106,113,113),NULL-- JyoB
---
[19:33:04] [INFO] testing SQLite
[19:33:04] [INFO] confirming SQLite
[19:33:04] [INFO] actively fingerprinting SQLite
[19:33:04] [INFO] the back-end DBMS is SQLite
back-end DBMS: SQLite
[19:33:04] [INFO] fetching tables for database: 'SQLite_masterdb'
<current>
[3 tables]
+-----------------+
| cars            |
| sqlite_sequence |
| users           |
+-----------------+
```

Fonte: Elaborado pelo autor

4.2 Identificando Explorações Simples

Como mostrado anteriormente, o envio do padrão `1 OR id=2` como valor para o parâmetro `id` retorna os dois carros, pois adiciona uma segunda verificação na cláusula `WHERE`. Caso o *firewall* de aplicação desenvolvido esteja protegendo essa API, é esperado que uma página de bloqueio seja retornada, ao invés dos dois carros cadastrados. Como o teste a seguir mostra, o firewall de aplicação conseguiu identificar a tentativa de exploração da API.

O parâmetro `-i` do comando `curl` o faz mostrar os cabeçalhos de resposta da requisição.

```
curl "http://0.0.0.0:8080/?id=1+OR+id=2" -i

HTTP/1.1 403 Forbidden
X-Xyz-Id: oJtEK3kUDuaoebg5VBjDCWlj
Date: Thu, 21 Nov 2024 13:42:54 GMT
Content-Length: 147
Content-Type: text/plain; charset=utf-8

```

```
Blocked due to malicious request detected!
Tokens found: 5
Your IP: 127.0.0.1:35916
Request ID: oJtEK3kUDuaoebg5VBjDCWlj
Time: 11-21-2024 10:42:54
```

Algumas informações sobre o ataque foram retornadas na requisição, primeiro há no cabeçalho o código de *status* 403 informando o bloqueio do acesso. No corpo da resposta é informado que foi identificado uma sequência de 5 tokens, que se refere ao parâmetro validado. Além disso, há o endereço IP do usuário e uma identificação da requisição. Na linha de *log* gerada pelo proxy reverso, também é possível identificar alguns dos dados mencionados:

```
2024/11/21 10:42:54 edp=http://0.0.0.0:8081/?id=1+OR+id=2 adr
    =127.0.0.1:35916 uri=/?id=1+OR+id=2 err="Exploitation attempt" tks=5
    rid=oJtEK3kUDuaoebg5VBjDCWlj
```

4.2.1 Teste SQLMap com WAF

A análise visa verificar como as três partes lidaram com o ataque, o SQLMap, o *proxy* reverso com o *firewall* de aplicação, e a API vulnerável.

Figura 12 – Tentativa de extração das tabelas com SQLMap

```
[15:43:56] [INFO] flushing session file
[15:43:56] [INFO] testing connection to the target URL
[15:43:56] [INFO] checking if the target is protected by some kind of WAF/IPS
[15:43:56] [CRITICAL] heuristics detected that the target is protected by some kind of WAF/IPS
are you sure that you want to continue with further target testing? [Y/n]

[15:44:00] [WARNING] please consider usage of tamper scripts (option '--tamper')
[15:44:00] [INFO] testing if the target URL content is stable
[15:44:00] [INFO] target URL content is stable
[15:44:00] [INFO] testing if GET parameter 'id' is dynamic
[15:44:00] [INFO] GET parameter 'id' appears to be dynamic
[15:44:00] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[15:44:00] [INFO] testing for SQL injection on GET parameter 'id'
[15:44:00] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[15:44:00] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[15:44:00] [INFO] testing 'MySQL ≥ 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[15:44:00] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[15:44:00] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[15:44:00] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[15:44:00] [INFO] testing 'Generic inline queries'
[15:44:00] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[15:44:00] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[15:44:00] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[15:44:00] [INFO] testing 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)'
[15:44:00] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[15:44:00] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[15:44:00] [INFO] testing 'Oracle AND time-based blind'
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of requests? [Y/n]

[15:44:05] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[15:44:06] [WARNING] GET parameter 'id' does not seem to be injectable
[15:44:06] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more tests. If you
suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--ra
ndom-agent'
[15:44:06] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 75 times

[*] ending @ 15:44:06 /2024-11-21/
```

Fonte: Elaborado pelo autor

A quarta linha da Figura 12 emite a linha "`[15:43:56] [CRITICAL] heuristics detected that the target is protected by some kind of WAF/IPS`", mencionando que um *firewall* de aplicação foi detectado via testes heurísticos.

Um ponto interessante é a linha que contém a seguinte mensagem: "`[15:44:00] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be`

`injectable`". Ela menciona que com todos os testes aplicados pelo SQLMap, uma vulnerabilidade que permite a injeção de SQL não foi identificada na aplicação testada. Então, a ferramenta passa a fazer testes mais agressivos para diferentes bancos de dados, mas mesmo assim sem sucesso.

As últimas linhas da Figura 12 mencionam que, de todas as requisições feitas para a aplicação, 75 retornaram código de *status* 403, foram bloqueadas pelo *firewall* de aplicação. No total, o SQLMap efetuou 78 requisições, e 3 delas não foram detectadas como tentativas de ataque. Essas requisições não possuem tokens no parâmetro `id`.

```
2024/11/21 15:43:56 edp=http://0.0.0.0:8081/?id=1 adr
    =192.168.15.117:47476 uri=/?id=1
2024/11/21 15:44:00 edp=http://0.0.0.0:8081/?id=1 adr
    =192.168.15.117:47506 uri=/?id=1
2024/11/21 15:44:00 edp=http://0.0.0.0:8081/?id=9554 adr
    =192.168.15.117:47514 uri=/?id=9554
```

Verificando os *logs* de acesso da API, do período do ataque, como mostrado na Figura 13, é possível ver que apenas os acessos não maliciosos, foram direcionados à API, os outros acessos qualificados como tentativa de exploração foram barrados.

Figura 13 – Logs de acesso da API sobre a tentativa de ataque com SQLMap

```
2024/11/21 15:43:51 q='SELECT brand, model FROM cars WHERE id=1;'
2024/11/21 15:43:56 q='SELECT brand, model FROM cars WHERE id=1;'
2024/11/21 15:44:00 q='SELECT brand, model FROM cars WHERE id=1;'
2024/11/21 15:44:00 q='SELECT brand, model FROM cars WHERE id=9554;'
```

Fonte: Elaborado pelo autor

4.2.2 Fuzzing Com Lista Padrões Maliciosos

Para este teste, foi utilizada uma lista contendo 68 padrões específicos para identificar vetores de injeção de SQL. Esses padrões foram enviados para a aplicação, e suas respostas analisadas. Em muitos casos, essas respostas facilitaram a identificação de manipulações inadequadas das interações do usuário, bem como a detecção de falhas que resultam no comprometimento ou na corrupção de consultas SQL. Foi esperado que todas as requisições feitas durante esse teste fossem bloqueadas.

A lista de padrões pode ser encontrada no apêndice.

Durante a aplicação do teste, foi percebido que mesmo utilizando uma consulta genérica para números inteiros no WAF, ainda foi possível enviar *strings*, sem que o WAF tratasse isso como uma tentativa de exploração, pelo envio de um tipo de dado diferente do esperado. Isso se dá pelo fato de, um inteiro e uma *string* serem considerados como apenas um token na expressão.

A seguir podem ser vistos os *logs* emitidos pela API, mostrando que os acessos chegaram à aplicação. É possível ver que diversos padrões de *string* não foram bloqueados.

```
2024/11/22 17:39:35 q='SELECT brand, model FROM cars WHERE id=‘‘;'
2024/11/22 17:39:35 q='SELECT brand, model FROM cars WHERE id='';'
2024/11/22 17:39:35 q='SELECT brand, model FROM cars WHERE id='=';'
2024/11/22 17:39:35 q='SELECT brand, model FROM cars WHERE id='LIKE';'
2024/11/22 17:39:35 q='SELECT brand, model FROM cars WHERE id=' or ";'
2024/11/22 17:39:35 q='SELECT brand, model FROM cars WHERE id="";'
```

A Tabela 3 mostra os testes que retornaram código de *status* 200, ou sucesso, junto do tamanho do corpo de resposta.

Padrão	**Código de *Status***	**Tamanho da Resposta em *bytes***
“	200	170
' '	200	157
'='	200	157
'LIKE'	200	157
' or "	200	183
" "	200	157

Tabela 3 – Padrões que geraram loops

Algumas requisições fizeram o *parser* do *firewall* de aplicação entrar em recursão sem fim (*loop*), o que fez com que essas requisições nunca fossem respondidas. Esse tipo de comportamento por parte do *firewall* de aplicação é prejudicial, pois consome os recursos do sistema onde está em execução até que haja intervenção. A Tabela 4 mostra as requisições que não foram respondidas por entrarem em recursão.

Padrão	**Código de *Status***	**Tamanho da Resposta em *bytes***
' OR 1 – -	0	0
"OR 1 = 1 – -	0	0
'=0–+	0	0
' AND id IS NULL; –	0	0
' GROUP BY columnnames having 1=1 –	0	0
' UNION SELECT sum(columnname) from tablename –	0	0

Tabela 4 – Requisições não respondidas

Além disso, durante os testes foi percebido que requisições contendo tokens não implementados no *parser* foram bloqueadas. Isso ocorre pelo fato desses tokens, serem interpretados como identificadores, gerando um erro de sintaxe pela forma conforme posicionados na consulta, ocasionando no bloqueio pela ocorrência do erro de sintaxe no lado do *firewall* de aplicação.

Com essa observação, foi percebido que não é necessário ter uma gramática completa implementada para verificar se os acessos possuem uma sequência de tokens nas delimitações

dessa gramática, pois apenas com a verificação de erro, é possível identificar a existência desses tokens. Portanto, não é necessário ter um analisador completo de SQL para identificar se a expressão está conforme a gramática, sendo que quando se valida pelo erro, isso trará apenas a informação de que há uma sequência de tokens quando, na verdade, deveria haver apenas um, no caso o identificador.

5 Conclusão

Esta monografia apresentou uma proposta de *firewall* de aplicação baseado em análise sintática, com o objetivo de melhorar a precisão na detecção de acessos maliciosos. O processo de análise foi estruturado em três etapas principais: primeiramente, o *firewall* recebia a requisição HTTP e decompunha seus argumentos; em seguida, era realizada a análise léxica desses dados; e, por fim, uma análise sintática avaliava se os argumentos da requisição formavam expressões consistentes com a gramática do SQL. Esse fluxo permitia identificar acessos potencialmente maliciosos.

A proposta demonstrou ser capaz de alcançar seu objetivo principal, conseguindo detectar padrões maliciosos destinados a explorar vulnerabilidades relacionadas à injeção de SQL e ao encadeamento de comandos em consultas construídas a partir de dados manipuláveis pelo usuário. No entanto, em seu estado atual, a solução descrita apresenta limitações que tornam sua implementação em ambiente real inviável, principalmente devido ao elevado consumo de recursos em determinados casos, o que pode comprometer o desempenho e a estabilidade do sistema.

Foi percebido durante testes que não é necessário ter a gramática completa implementada. Quando o bloqueio se dá pela ocorrência de erros de sintaxe do lado do *firewall* de aplicação, pelo fato da gramática limitada, o analisador seguindo as regras da gramática, mas não tem necessariamente uma variedade de comandos implementados, isso ainda vai denunciar a existência de mais tokens do que o esperado na expressão analisada.

5.1 Trabalhos Futuros

Por conta de limitações impostas no projeto para poder ser desenvolvido no tempo proposto, e comportamentos da solução durante testes, há algumas sugestões de melhorias e adições que podem ser úteis em trabalhos futuros.

- Analisar todas as ocorrências de recursão e traçar estratégias para impedir que isso ocorra: durante testes foi percebido que alguns padrões fazem o analisador desenvolvido entrar em *loop*, e isso irá ocorrer até que haja uma intervenção externa no processo no *firewall* de aplicação. Para que a solução seja estável e segura, é necessário que não haja esse tipo de comportamento que é danoso aos recursos do sistema e a experiência do usuário.

- Análise do uso de recursos para entender como a escala nos acessos afeta o uso de recursos da solução: É imprescindível que, em um ambiente que receba muitas inte-

rações de usuário, a solução de segurança não seja capaz de impactar negativamente na experiência adicionando uma latência considerável. É importante realizar testes de estresse para e simulações para entender como a solução usa a CPU e memória volátil em diferentes casos, tanto acessos normais quanto acessos que contenham padrões maliciosos.

- Traçar testes comparativos com outras soluções de mercado: Como mencionado na revisão bibliográfica, existem diversas soluções comerciais e de código aberto no mercado, e gera métricas úteis e importantes ter uma comparação entre essas soluções e o software neste documento descrito.

Referências

4 coreruleset. Owasp crs project. 2024. Disponível em: <https://coreruleset.org/>. Acesso em: 27 maio 2024. Citado na página 34.

AKAMAI. Improving web application security: The akamai approach to waf. 2015. Disponível em: <https://www.akamai.com/site/pt/documents/white-paper/improving-web-application-security-the-akamai-approach-to-waf-white-paper.pdf>. Acesso em: 27 maio 2024. Citado 4 vezes nas páginas 32, 33, 34 e 35.

AKAMAI. Akamai cdn. 2024. Disponível em: <https://www.akamai.com/pt>. Acesso em: 27 maio 2024. Citado na página 31.

BELLO, M. S. A. B. I. S. I. abubakar; Bello shamsudden kalgo e Abdul-azeez M. Web application firewall: Review. 2020. Disponível em: <http://www.ijciras.com/PublishedPaper/IJCIRAS1243.pdf>. Acesso em: 30 abr 2024. Citado na página 17.

BELLOVIN, S. M. Security problems in the tcp/ip protocol suite. 1989. Citado na página 19.

BOTEZATU, L. Xss attack: 3 real life attacks and code examples. 2013. Disponível em: <https://www.bitdefender.com/blog/hotforsecurity/yahoo-accounts-hijacked-via-xss-type-attack/>. Acesso em: 26 maio 2024. Citado na página 28.

CISCO. What is a cyberattack? 2024. Disponível em: <https://www.cisco.com/c/en/us/products/security/common-cyberattacks.html>. Acesso em: 26 maio 2024. Citado na página 24.

CLOUDFLARE. O que é um firewall? como funcionam os firewalls de rede. 2018. Disponível em: <https://www.cloudflare.com/pt-br/learning/security/what-is-a-firewall/>. Acesso em: 26 maio 2024. Citado na página 30.

CLOUDFLARE. O que é segurança de aplicativos web? 2022. Disponível em: <https://www.cloudflare.com/pt-br/learning/security/what-is-web-application-security/>. Acesso em: 26 maio 2024. Citado na página 28.

CLOUDFLARE. Cloudflare cdn. 2024. Disponível em: <https://www.cloudflare.com/pt-br/>. Acesso em: 27 maio 2024. Citado na página 31.

DELAMARO, M. *Como Construir um Compilador Utilizando Ferramentas Java*. [S.l.]: Novatec, 2014. Citado 3 vezes nas páginas 36, 37 e 38.

DESCARTES, N. Writing a lexical analyzer for sql in csharp. 2024. Disponível em: <https://github.com/combinators/websecbench>. Acesso em: 27 maio 2024. Citado na página 36.

E. E. Schultz Jr; D. S. Brown; T. A. Longstaff. *Responding to computer security incidents: Guidelines for incident handling*. [S.l.]: Lawrence Livermore National Lab, 1990. Citado 2 vezes nas páginas 19 e 20.

ECOTRUST. Os impactos financeiros dos ataques cibernéticos nas empresas. 2023. Disponível em: <https://blog.ecotrust.io/impactos-financeiros-ataques-ciberneticos/>. Acesso em: 26 maio 2024. Citado na página 27.

FAROOQ, R. A. H. A. S. S. M. M. A. H. Critical analysis on web application firewall solutions. p. 1–6, 2013. Disponível em: <https://ieeexplore.ieee.org/document/6513431>. Acesso em: 26 maio 2024. Citado 2 vezes nas páginas 31 e 33.

GUERRERO-SAADE, J. A. Hacktivism and state-sponsored knock-offs | attributing deceptive hack-and-leak operations. 2022. Disponível em: <https://www.sentinelone.com/labs/hacktivism-and-state-sponsored-knock-offs-attributing-deceptive-hack-and-leak-operations/>. Acesso em: 26 maio 2024. Citado na página 27.

GUNAWAN, H. F. B. S. F. E. Prevention structured query language injection using regular expression and escape string. 2018. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1877050918315114>. Acesso em: 6 maio 2024. Citado na página 31.

GUPTA, S. G. B. B. Cross-site scripting (xss) attacks and defense mechanisms: classification and state-of-the-art. *International Journal of System Assurance Engineering and Management*, 2015. Disponível em: <https://link.springer.com/article/10.1007/S13198-015-0376-0>. Acesso em: 26 maio 2024. Citado 2 vezes nas páginas 22 e 28.

HACKER1 TOP 10. 2023. Disponível em: <https://www.hackerone.com/top-ten-vulnerabilities>. Acesso em: 30 abr 2024. Citado na página 17.

HARRIS, B.; HUNT, R. Tcp/ip security threats and attack methods. *Computer Communications*, v. 22, n. 10, p. 885–897, 1999. ISSN 0140-3664. Disponível em: <https://www.sciencedirect.com/science/article/pii/S014036649900064X>. Citado na página 19.

HEALTH intraprise. Cybersecurity nightmares: The cost of healthcare cyberattacks in 2023. 2024. Disponível em: <https://intraprisehealth.com/the-cost-of-cyberattacks-in-healthcare/>. Acesso em: 26 maio 2024. Citado 2 vezes nas páginas 27 e 28.

IBM. Como solucionar problemas de falsos positivos e falsos negativos no waf? 2020. Disponível em: <https://cloud.ibm.com/docs/cis?topic=cis-waf-false-pos-neg>. Acesso em: 27 maio 2024. Citado na página 33.

IBM. Why do cyberattacks happen? 2024. Disponível em: <https://www.ibm.com/topics/cyber-attack>. Acesso em: 26 maio 2024. Citado na página 26.

ILYIN, S. A7: Cross-site scripting (xss) 2017 owasp. 2024. Disponível em: <https://www.wallarm.com/what/a7-cross-site-scripting-xss-2017-owasp>. Acesso em: 26 maio 2024. Citado 2 vezes nas páginas 22 e 23.

IMPERVA. Imperva cdn. 2024. Disponível em: <https://www.imperva.com/br/>. Acesso em: 27 maio 2024. Citado na página 31.

ISO. Information technology — database languages sql. 2023. Disponível em: <https://www.iso.org/standard/76583.html>. Acesso em: 11 novembro 2024. Citado 2 vezes nas páginas 42 e 44.

ITEAM. Entenda o que é vulnerabilidade de segurança e quais são as mais comuns. 2020. Disponível em: <https://it-eam.com/entenda-o-que-e-vulnerabilidade-de-seguranca-e-quais-sao-as-mais-comuns/>. Acesso em: 19 maio 2024. Citado na página 19.

ITRC, I.-T.-R.-C. Identity theft resource center's 2022 annual data breach report reveals near-record number of compromises. 2023. Disponível em: <https://www.idtheftcenter.org/post/2022-annual-data-breach-report-reveals-near-record-number-compromises/>. Acesso em: 26 maio 2024. Citado 2 vezes nas páginas 27 e 28.

ITRC, I.-T.-R.-C. Sql injection. 2023. Disponível em: <https://portswigger.net/web-security/sql-injection>. Acesso em: 26 maio 2024. Citado 3 vezes nas páginas 23, 24 e 29.

MARIANNE, M. B. A. Evaluating the modsecurity web application firewall against sql injection attacks. In: . [S.l.: s.n.], 2020. p. 1–6. Acesso em: 26 maio 2024. Citado na página 33.

MARIMUTHU, K. P. M. A. K. M. An efficient technique for preventing sql injection attack using pattern matching algorithm. 2013. Disponível em: <https://ieeexplore.ieee.org/abstract/document/6528551>. Acesso em: 6 maio 2024. Citado na página 26.

MCHENRY, B. A. What makes a waf advanced? 2018. Disponível em: <https://www.f5.com/pt_br/company/blog/what-makes-a-waf-advanced>. Acesso em: 26 maio 2024. Citado na página 30.

MCNALLY, R. et al. Fuzzing: The state of the art. Citeseer, 2012. Disponível em: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=d5524b450929c6d7ea9b6926ba060661cae07009>. Acesso em: 19 Janeiro 2025. Citado na página 51.

M.E, S. D. S. P. H. C. The code sanitizer: Regular expression based prevention of content injection attacks. *International Journal of Computer Trends and Technology (IJCTT)*, 2016. Disponível em: <https://www.ijcttjournal.org/2016/Volume35/number-1/IJCTT-V35P104.pdf>. Acesso em: 26 maio 2024. Citado na página 29.

Medeiros, Iberia; Beatriz, Miguel; Neves, Nuno; Correia, Miguel. Demonstrating a Tool for Injection Attack Prevention in MySQL. Institute of Electrical and Electronics Engineers Inc., p. 551–558, 8 2017. Citado na página 29.

MITRE. Cwe-20: Improper input validation. 2006. Disponível em: <https://cwe.mitre.org/data/definitions/20.html>. Acesso em: 26 maio 2024. Citado na página 21.

MITRE. Cwe-778: Insufficient logging. 2009. Disponível em: <https://cwe.mitre.org/data/definitions/778.html>. Acesso em: 26 maio 2024. Citado na página 24.

MITRE. Capec-63: Cross-site scripting (xss). 2014. Disponível em: <https://capec.mitre.org/data/definitions/63.html>. Acesso em: 26 maio 2024. Citado 2 vezes nas páginas 22 e 23.

MODSECURITY. Modsecurity web application firewall. 2024. Disponível em: <https://www.modsecurity.org/>. Acesso em: 26 maio 2024. Citado 3 vezes nas páginas 31, 32 e 33.

MORADOV, O. Xss attack: 3 real life attacks and code examples. 2022. Disponível em: <https://brightsec.com/blog/xss-attack/>. Acesso em: 26 maio 2024. Citado na página 28.

NICYBER. Cyber threats. 2024. Disponível em: <https://www.nicybersecuritycentre.gov.uk/cyber-threats>. Acesso em: 26 maio 2024. Citado na página 26.

OUISSEM, F. Spheres: an efficient server-side web application protection system. *International Journal of Information and Computer Security*, v. 11, p. 33, 01 2019. Disponível em: <https://www.researchgate.net/publication/330055462_SPHERES_an_efficient_server-side_web_application_protection_system>. Acesso em: 27 maio 2024. Citado 2 vezes nas páginas 34 e 35.

OWASP. A09:2021 – falhas de registro e monitoramento de segurança. 2021. Disponível em: <https://owasp.org/Top10/pt_BR/A09_2021-Security_Logging_and_Monitoring_Failures/>. Acesso em: 26 maio 2024. Citado na página 24.

OWASP. Vulnerabilities. 2023. Disponível em: <https://owasp.org/www-community/vulnerabilities/>. Acesso em: 19 maio 2024. Citado 2 vezes nas páginas 19 e 20.

OWASP TOP 10. 2021. Disponível em: <https://owasp.org/www-project-top-ten/>. Acesso em: 30 abr 2024. Citado 4 vezes nas páginas 17, 22, 23 e 35.

PALOALTO. 2024 unit 42 incident response report: Navigating the shift in cybersecurity threat tactics. 2024. Disponível em: <https://unit42.paloaltonetworks.com/unit42-incident-response-report-2024-threat-guide/>. Acesso em: 26 maio 2024. Citado na página 25.

RESEARCH, C. Check point research reports a 38 por ceinto increase in 2022 global cyberattacks. 2023. Disponível em: <https://blog.checkpoint.com/2023/01/05/38-increase-in-2022-global-cyberattacks/>. Acesso em: 26 maio 2024. Citado na página 27.

RIZKI, M. et al. Improving security of web-based application using modsecurity and reverse proxy in web application firewall. 2020. Disponível em: <https://www.researchgate.net/publication/347802820_Improving_Security_of_Web-Based_Application_Using_ModSecurity_and_Reverse_Proxy_in_Web_Application_Firewall>. Acesso em: 30 abril 2024. Citado na página 31.

SHAHRIAR, V. C. H. Web application firewall: Network security models and configuration. 2018. Disponível em: <https://ieeexplore.ieee.org/document/8377769>. Acesso em: 6 maio 2024. Citado na página 17.

SIMON, K. D. M. W. L. *The Art of Deception*. [S.l.]: xxx, 2001. Citado na página 20.

STAMPAR, B. D. A. G. M. Automatic sql injection and database takeover tool. 2006. Disponível em: <https://sqlmap.org/>. Acesso em: 20 novembro 2024. Citado na página 52.

STARLIN, G. *TCP/IP Internet - Intranet - Extranet*. [S.l.]: xxx, 1998. Citado na página 19.

SYNACK. Improper input validation. 2024. Disponível em: <https://learn.snyk.io/lesson/improper-input-validation/>. Acesso em: 26 maio 2024. Citado na página 21.

SYNC, D. Data hygiene: How to make your data accurate and actionable. 2024. Disponível em: <https://deepsync.com/data-hygiene/>. Acesso em: 26 maio 2024. Citado na página 29.

TAMAE, G. A. J. B. R. Y. Uma breve introduÇÃo ao estudo e implementaÇÃo de compiladores. 2008. Disponível em: <http://faef.revista.inf.br/imagens_arquivos/arquivos_destaque/mPd7q0kH4YgxlRK_2013-5-28-10-54-8.pdf>. Acesso em: 27 maio 2024. Citado 2 vezes nas páginas 36 e 38.

THOMAS, S. W. L. T. X. On automated prepared statement generation to remove sql injection vulnerabilities. 2009. Disponível em: <https://www.sciencedirect.com/science/article/abs/pii/S0950584908001110>. Acesso em: 26 maio 2024. Citado na página 29.

ULLMAN, A. V. A. M. S. L. R. S. J. D. *Compiladores - Princípios Técnicas e Ferramentas*. [S.l.]: LTC - Livros Técnicos e Cientificos, 1995. Citado 3 vezes nas páginas 35, 36 e 38.

WEBSECBENCH. Automatic generation of owasp-like websecurity benchmark. 2019. Disponível em: <https://github.com/combinators/websecbench>. Acesso em: 27 maio 2024. Citado na página 35.

WEI, K.; MUTHUPRASANNA, M.; KOTHARI, S. *Preventing SQL Injection Attacks in Stored Procedures*. 2006. Disponível em: <https://ieeexplore.ieee.org/abstract/document/1615052>. Acesso em: 12 maio 2024. Citado 2 vezes nas páginas 25 e 29.

APÊNDICE A – Lista de Padrões

A.1 Apendice A - Lista para Teste de *Fuzzing*

A lista a seguir foi utilizada no teste de *fuzzing*, para a verificação de padrões que pudessem gerar comportamentos inesperados no WAF.

```
'
''
`
``
,
"
""
/
//
\
\\
;
' or "
-- or #
' OR '1
' OR 1 -- -
" OR "" = "
" OR 1 = 1 -- -
' OR '' = '
'='
'LIKE'
'=0--+
 OR 1=1
' OR 'x'='x
' AND id IS NULL; --
''''''''''''UNION SELECT '2
%00
/*   */
+
||
%
@variable
@@variable
#
AND 1
AND 0
AND true
```

```
38 AND false
39 1-false
40 1-true
41 1*56
42 -2
43 1' ORDER BY 1--+
44 1' ORDER BY 2--+
45 1' ORDER BY 3--+
46 1' ORDER BY 1,2--+
47 1' ORDER BY 1,2,3--+
48 1' GROUP BY 1,2,--+
49 1' GROUP BY 1,2,3--+
50 ' GROUP BY columnnames having 1=1 --
51 -1' UNION SELECT 1,2,3--+
52 ' UNION SELECT sum(columnname ) from tablename --
53 -1 UNION SELECT 1 INTO @,@
54 -1 UNION SELECT 1 INTO @,@,@
55 1 AND (SELECT * FROM Users) = 1
56 ' AND MID(VERSION(),1,1) = '5';
57 ' and 1 in (select min(name) from sysobjects where xtype = 'U' and name
     > '.') --
58 ,(select * from (select(sleep(10)))a)
59 %2c(select%20*%20from%20(select(sleep(10)))a)
60 ';WAITFOR DELAY '0:0:30'--
61 Comments:
62 #
63 /*
64 -- -
65 ;%00
66 `
```